

单位代码： 10293 密 级：

南京邮电大学
专 业 学 位 硕 士 论 文



论文题目： Lark: 基于强化学习的多信号融合自适应网络拥塞控制算法

学 号	<u>1224045520</u>
姓 名	<u>杭天铖</u>
导 师	<u>王俊昌教授</u>
专业学位类别	<u>电子信息</u>
类 型	<u>全日制</u>
专业（领域）	<u>计算机技术</u>
论文提交日期	<u>2026 年 3 月</u>

摘要

随着云计算、大数据和人工智能技术的快速发展，现代数据中心网络规模呈指数级增长，对传输控制协议（TCP）拥塞控制机制的性能提出了前所未有的挑战。传统基于丢包的拥塞控制算法（如 TCP NewReno、CUBIC）存在拥塞信号滞后、高带宽延迟积（Bandwidth-Delay Product, BDP）网络收敛缓慢、无法区分拥塞程度等固有缺陷；而基于延迟的算法（如 TCP Vegas、BBR）则面临往返时延（Round-Trip Time, RTT）测量不稳定、与丢包算法共存时公平性差等问题。本文提出 Lark 算法——一种基于强化学习的多信号融合自适应网络拥塞控制方法。该算法名称寓意“云雀”，象征着高吞吐量（High Throughput）与低延迟（Low Latency）的设计目标，如同云雀般轻盈敏捷、反应迅速。

Lark 算法的核心创新包括：（1）设计了包含 15 个参数的完整观测空间，首次将显式拥塞通知（Explicit Congestion Notification, ECN）状态、拥塞控制状态机（Congestion Algorithm State）状态、拥塞事件类型等丰富的协议栈信息纳入强化学习状态表示；（2）提出多信号融合拥塞检测方法，采用分层优先级机制综合分析丢包信号、ECN 信号、拥塞状态信号，并通过频率过滤策略区分持续性拥塞和瞬态扰动；（3）设计差异化拥塞响应机制，针对不同拥塞类型采用不同的窗口保留因子（Window Retention Factor）——丢包 0.70、ECN 标记 0.85、超时 0.50，并引入连续缩减保护机制（最多连续 3 次缩减后保持窗口），充分区分拥塞信号的性质和严重程度；（4）实现强化学习驱动的自适应参数调优，根据 RTT 膨胀比、奖励信号的指数移动平均（Exponential Moving Average, EMA）、连续增长趋势等多因素动态调整乘性增加因子 α （取值范围 $[1.05, 1.50]$ ，基准值 1.20）；（5）采用融合控制策略 $V_t = \max(\alpha \times BDP, W) + \gamma \times MSS$ （ $\gamma = 2.0$ ），将基于 BDP 窗口化最大值过滤（Windowed Max-filter）估计的速率控制与基于当前窗口的保守控制相结合，并引入管道利用率感知的加速增长机制。

本文基于 ns-3 网络仿真平台和 ns3-gym/OpenGym 强化学习框架实现了 Lark 算法，采用 ZeroMQ 进程间通信实现仿真进程与决策进程的高效协作，并采用 MTP-Interface 多线程并行化仿真架构加速训练验证。在涵盖机架内高速网络、不同过载比场景、跨 Pod 网络和跨数据中心广域网（Wide Area Network, WAN）等 9 种典型数据中心网络场景的大量实验中，Lark 算法相比传统算法展现出显著优势：在数据中心内部场景下，Lark 实现了与传统最优算法（NewReno/CUBIC）相当的吞吐量（差距 $<1\%$ ），同时将延迟降低了 70% 至 81%（从 1.9~2.8ms 降至 0.436~0.837ms）；在跨 Pod 和 WAN 场景下，Lark 的吞吐量与传统算法持平，带宽利用率均超过 99%。实验结果表明，Lark 算法成功实现了“高吞吐、低延迟”的设计目标，为数据中心网络拥塞控制提供了一种高效、自适应的解决方案。

关键词：拥塞控制，强化学习，多信号融合，显式拥塞通知，自适应参数调优，数据中心网络

Abstract

With the rapid development of cloud computing, big data, and artificial intelligence technologies, modern data center networks have experienced exponential growth in scale, posing unprecedented challenges to the performance of Transmission Control Protocol (TCP) congestion control mechanisms. Traditional loss-based congestion control algorithms (such as TCP NewReno and CUBIC) suffer from inherent defects including delayed congestion signals, slow convergence in high Bandwidth-Delay Product (BDP) networks, and inability to distinguish congestion severity. Meanwhile, delay-based algorithms (such as TCP Vegas and BBR) face issues like unstable Round-Trip Time (RTT) measurements and poor fairness when coexisting with loss-based algorithms.

This paper proposes Lark, a reinforcement learning-based multi-signal fusion adaptive network congestion control algorithm. The name “Lark” symbolizes the design goals of High Throughput and Low Latency, like a lark that is light, agile, and responsive. The core innovations of Lark include: (1) a complete observation space containing 15 parameters, incorporating Explicit Congestion Notification (ECN) status, Congestion Algorithm (CA) state machine status, and congestion event types into the reinforcement learning state representation for the first time; (2) a multi-signal fusion congestion detection method using hierarchical priority mechanisms to comprehensively analyze packet loss signals, ECN signals, and congestion state signals, with frequency filtering to distinguish persistent congestion from transient disturbances; (3) a differentiated congestion response mechanism with different window retention factors for different congestion types (packet loss: 0.70, ECN: 0.85, timeout: 0.50), along with consecutive decrease protection (holding window after 3 consecutive reductions); (4) reinforcement learning-driven adaptive parameter tuning based on RTT inflation ratio, reward signal EMA, and consecutive growth trends, dynamically adjusting the multiplicative increase factor $\alpha \in [1.05, 1.50]$ with base value 1.20; (5) a fusion control strategy $V_t = \max(\alpha \times BDP, W) + \gamma \times MSS$ ($\gamma = 2.0$) combining windowed max-filter BDP estimation with current window-based conservative control and pipeline utilization-aware acceleration.

We implemented Lark based on the ns-3 network simulation platform and ns3-gym/OpenGym reinforcement learning framework, with ZeroMQ inter-process communication and MTP-Interface multi-threaded parallel simulation architecture to accelerate training and validation. Extensive experiments across 9 typical data center network scenarios demonstrate significant advantages: in data center internal scenarios, Lark achieved throughput comparable to traditional optimal algorithms (NewReno/CUBIC) with less than 1% gap, while reducing latency by 70% to 81% (from 1.9~2.8ms to 0.436~0.837ms); in cross-Pod and WAN scenarios, Lark matched traditional algorithms in through-

put with bandwidth utilization exceeding 99%.

Key Words: Congestion Control, Reinforcement Learning, Multi-signal Fusion, Explicit Congestion Notification, Adaptive Parameter Tuning, Data Center Network

目 录

摘要	I
Abstract	II
插图索引	IX
表格索引	X
专用术语注释表	XI
第一章 绪论	1
1.1 研究背景与动机	1
1.2 研究目标与主要贡献	2
1.3 论文组织结构	5
第二章 相关工作	6
2.1 传统拥塞控制算法	6
2.1.1 基于丢包的拥塞控制算法	6
2.1.2 基于延迟的拥塞控制算法	8
2.1.3 混合拥塞控制算法	9
2.2 显式拥塞通知机制	10
2.2.1 ECN 工作原理	10
2.2.2 ECN 相关拥塞控制算法	10
2.3 基于机器学习的拥塞控制方法	12
2.3.1 监督学习方法	12
2.3.2 强化学习方法	12
2.4 数据中心网络特性分析	13
2.4.1 网络拓扑特性	13
2.4.2 流量特性分析	13
2.4.3 应用延迟需求分析	14
2.4.4 网络设备特性分析	14
2.5 拥塞控制的理论基础	15
2.5.1 网络流量模型	15
2.5.2 排队论分析	16
2.5.3 控制论视角	16
2.6 强化学习在网络中的应用综述	17
2.6.1 路由优化	17
2.6.2 资源分配	18
2.6.3 安全防护	18
2.6.4 与 Lark 的关系	18
2.7 本章小结	19
第三章 Lark 算法设计与实现	20
3.1 系统架构概述	20
3.2 强化学习问题建模	21
3.2.1 状态空间设计理论	21
3.2.2 动作空间设计理论	22
3.2.3 奖励函数设计理论	22
3.2.4 策略表示理论	23

3.3	15 维观测空间详细设计	23
3.3.1	连接标识参数（索引 0、3）	23
3.3.2	时间信息参数（索引 1、2）	24
3.3.3	窗口状态参数（索引 4、5）	24
3.3.4	传输性能参数（索引 6、7、8）	24
3.3.5	延迟测量参数（索引 9、10）	25
3.3.6	回调类型参数（索引 11）	25
3.3.7	拥塞控制状态参数（索引 12、13、14）	26
3.4	多信号融合拥塞检测算法	27
3.4.1	拥塞信号分类	27
3.4.2	分层优先级机制	27
3.4.3	频率过滤策略	28
3.4.4	完整拥塞检测算法	28
3.5	差异化拥塞响应机制	29
3.5.1	窗口保留因子设计	29
3.5.2	差异化响应的理论依据	30
3.5.3	慢启动阈值更新	30
3.5.4	差异化拥塞响应算法	31
3.6	强化学习驱动的自适应参数调优	31
3.6.1	乘性增加因子的作用	31
3.6.2	多因素调整机制	32
3.6.3	参数边界约束	33
3.6.4	自适应参数调优算法	33
3.7	融合控制策略	34
3.7.1	带宽延迟积估计	34
3.7.2	慢启动阶段策略	34
3.7.3	拥塞避免阶段策略	35
3.7.4	窗口边界约束	35
3.7.5	融合控制策略算法	36
3.8	每流独立状态管理	36
3.8.1	状态数据结构	36
3.8.2	状态查找与更新	37
3.8.3	状态生命周期管理	38
3.9	实现细节与优化技术	38
3.9.1	系统架构设计	38
3.9.2	核心算法伪代码	38
3.9.3	数据结构设计	38
3.9.4	性能优化技术	40
3.9.5	多线程并行化架构	40
3.9.6	正确性保证机制	40
3.10	本章小结	41
第四章	实验评估	43
4.1	实验环境配置	43
4.1.1	硬件环境	43
4.1.2	软件环境	43
4.1.3	网络拓扑	43
4.1.4	仿真参数	44

4.2	评测指标	44
4.2.1	吞吐量 (Throughput)	44
4.2.2	延迟 (Latency)	45
4.2.3	公平性 (Fairness)	45
4.2.4	丢包率 (Loss Rate)	45
4.3	对比算法	46
4.3.1	TCP NewReno	46
4.3.2	TCP CUBIC	46
4.3.3	TCP BBR	46
4.4	实验场景设计	46
4.4.1	场景一：机架内 10Gbps 高速场景	46
4.4.2	场景二：4:1 过载比 10Gbps 场景	46
4.4.3	场景三：4:1 过载比 40Gbps 场景	47
4.4.4	场景四：中度拥塞场景	47
4.4.5	场景五：重度拥塞场景	47
4.4.6	场景六：跨 Pod 10Gbps 场景	47
4.4.7	场景七：跨 Pod 20Gbps 场景	47
4.4.8	场景八：跨数据中心 WAN 场景	47
4.4.9	场景九：对称低带宽场景	47
4.5	实验方法论	48
4.5.1	实验设计原则	48
4.5.2	数据采集方法	48
4.5.3	统计分析方法	48
4.6	实验结果分析	48
4.6.1	场景一结果：机架内 10Gbps 高速场景	48
4.6.2	场景二结果：4:1 过载比 10Gbps 场景	49
4.6.3	场景三结果：4:1 过载比 40Gbps 场景	49
4.6.4	场景四结果：中度拥塞场景	49
4.6.5	场景五结果：重度拥塞场景	50
4.6.6	场景六结果：跨 Pod 10Gbps 场景	50
4.6.7	场景七结果：跨 Pod 20Gbps 场景	50
4.6.8	场景八结果：跨数据中心 WAN 场景	50
4.6.9	场景九结果：对称低带宽场景	51
4.7	综合分析	51
4.7.1	吞吐量分析	51
4.7.2	延迟分析	51
4.7.3	带宽利用率分析	52
4.7.4	收敛特性分析	52
4.7.5	公平性分析	53
4.7.6	混合流量分析	53
4.7.7	算法优势与局限性总结	53
4.8	本章小结	54

第五章 讨论	55
5.1 设计考量	55
5.1.1 15 维观测空间的设计原理	55
5.1.2 差异化窗口保留因子的选择	55
5.1.3 自适应参数调优的设计	56
5.2 理论分析	56
5.2.1 收敛性分析	56
5.2.2 公平性分析	56
5.2.3 复杂度分析	56
5.3 局限性与改进方向	56
5.3.1 高延迟场景优化	56
5.3.2 参数自动调优	57
5.3.3 多目标优化	57
5.3.4 安全性考虑	57
5.4 与相关工作的比较	57
5.4.1 与 Aurora 的比较	57
5.4.2 与 DCTCP 的比较	57
5.4.3 与 BBR 的比较	57
5.4.4 与 PCC 的比较	58
5.5 敏感性分析	58
5.5.1 窗口保留因子敏感性	58
5.5.2 连续缩减保护阈值敏感性	58
5.5.3 RTT 膨胀比阈值敏感性	59
5.5.4 历史缓冲区大小敏感性	59
5.6 计算开销分析	59
5.6.1 决策延迟	59
5.6.2 内存占用	59
5.6.3 CPU 利用率	60
5.7 理论分析深入探讨	60
5.7.1 多信号融合的信息论基础	60
5.7.2 差异化响应的最优控制分析	61
5.7.3 自适应参数调优的强化学习视角	61
5.7.4 收敛性的 Lyapunov 分析	61
5.7.5 公平性的博弈论分析	62
5.8 本章小结	62
第六章 结论与展望	63
6.1 研究总结	63
6.2 研究展望	65
6.2.1 WAN 场景延迟优化	65
6.2.2 深度强化学习增强	66
6.2.3 多目标优化扩展	66
6.2.4 真实网络部署与验证	66
6.2.5 安全性与鲁棒性研究	67
6.2.6 跨层优化研究	67
6.2.7 异构网络适配	68
6.2.8 与新兴传输协议的融合	68
6.2.9 绿色网络与能效优化	69
6.2.10 可解释人工智能在网络中的应用	69

6.3 应用场景展望	69
6.3.1 云计算数据中心	70
6.3.2 边缘计算	70
6.3.3 内容分发网络	70
6.3.4 高性能计算	70
6.3.5 金融交易系统	71
6.4 研究意义与社会价值	71
6.4.1 理论意义	71
6.4.2 实际应用价值	72
6.4.3 社会影响	72
参考文献	73
附录 A 程序清单	75
附录 B 攻读专业学位硕士期间撰写的论文	76
附录 C 攻读专业学位硕士期间申请的专利	77
附录 D 攻读专业学位硕士期间参加的科研项目	78
致谢	79

插图索引

图 1.1	Lark 算法核心贡献概览	3
图 2.1	拥塞控制算法分类体系	6
图 2.2	ECN 信号传递工作流程	11
图 3.1	Lark 系统架构	20
图 4.1	哑铃型（Dumbbell）实验网络拓扑	44
图 4.2	数据中心内部场景延迟对比（不含 BBR）	52

表格索引

表 2.1 代表性拥塞控制算法综合对比 18

表 3.1 Lark 15 维观测空间参数定义 24

表 3.2 差异化窗口保留因子 29

表 4.1 默认仿真参数 44

表 4.2 场景一实验结果 48

表 4.3 场景二实验结果 49

表 4.4 场景三实验结果 49

表 4.5 场景四实验结果 49

表 4.6 场景五实验结果 50

表 4.7 场景六实验结果 50

表 4.8 场景七实验结果 50

表 4.9 场景八实验结果 51

表 4.10 场景九实验结果 51

表 4.11 吞吐量综合对比（单位：Mbps） 51

表 4.12 延迟综合对比（单位：ms） 52

表 4.13 带宽利用率对比（%） 53

表 4.14 收敛时间对比（场景二） 53

表 4.15 公平性测试结果 53

表 4.16 混合流量测试结果 54

表 5.1 Lark 与 Aurora 的详细对比 57

表 5.2 Lark 与 DCTCP 的详细对比 58

表 5.3 Lark 与 BBR 的详细对比 58

表 5.4 丢包保留因子敏感性分析（场景二） 58

表 5.5 ECN 保留因子敏感性分析（场景二） 59

表 5.6 连续缩减保护阈值敏感性分析（场景二） 59

表 5.7 RTT 膨胀比阈值敏感性分析（场景二） 59

表 5.8 历史缓冲区大小敏感性分析（场景二） 60

表 5.9 决策延迟测量结果 60

表 5.10 内存占用分析 60

表 5.11 CPU 利用率测量结果 61

专用术语注释表

符号说明

$cwnd$	拥塞窗口 (Congestion Window), TCP 发送端在未收到确认的情况下最多可以发送的数据量 (字节)
$ssthresh$	慢启动阈值 (Slow Start Threshold), 区分慢启动和拥塞避免阶段的临界值 (字节)
α	乘性增加因子 (Multiplicative Increase Factor), 控制拥塞避免阶段窗口增长速度, 取值范围 $[1.05, 1.50]$, 基准值 1.20
γ	加性增加因子 (Additive Increase Factor), 控制窗口的加性增量, 默认值为 2.0
ρ	窗口保留因子 (Window Retention Factor), 拥塞响应后新窗口与原窗口的比值
ρ_{loss}	丢包保留因子, 默认值 0.70
ρ_{ecn}	ECN 保留因子, 默认值 0.85
$\rho_{timeout}$	超时保留因子, 默认值 0.50
η	奖励信号指数移动平均 (EMA) 衰减系数, 默认值 0.1
BDP	带宽延迟积 (Bandwidth-Delay Product), 网络链路能够容纳的最大在途数据量 (字节)
MSS	最大报文段长度 (Maximum Segment Size), 典型值为 1460 字节
RTT	往返时延 (Round-Trip Time), 数据包从发送端到接收端再返回的总时间
$W(t)$	时刻 t 的拥塞窗口大小
W_{max}	上次拥塞事件前的最大窗口大小
K	CUBIC 算法中恢复到 W_{max} 所需的时间
C	链路容量或 CUBIC 缩放因子
β	窗口缩减因子
δ	延迟惩罚权重参数
λ	丢包惩罚权重参数

H	Hurst 参数，描述流量自相似性
$\bar{Q}$	平均队列长度
$\bar{W}$	平均等待时间
τ	反馈延迟（约等于 RTT）

缩略词

TCP	传输控制协议 (Transmission Control Protocol)
ECN	显式拥塞通知 (Explicit Congestion Notification)
RTT	往返时延 (Round-Trip Time)
BDP	带宽延迟积 (Bandwidth-Delay Product)
MDP	马尔可夫决策过程 (Markov Decision Process)
RL	强化学习 (Reinforcement Learning)
AIMD	加性增乘性减 (Additive Increase Multiplicative Decrease)
MSS	最大报文段长度 (Maximum Segment Size)
DCTCP	数据中心 TCP (Data Center TCP)
BBR	瓶颈带宽和往返传播时间 (Bottleneck Bandwidth and Round-trip propagation time)
CUBIC	三次函数拥塞控制算法
ACK	确认 (Acknowledgment)
SACK	选择性确认 (Selective Acknowledgment)
CE	拥塞经历 (Congestion Experienced)
ECE	ECN 回显 (ECN-Echo)
CWR	拥塞窗口缩减 (Congestion Window Reduced)
RED	随机早期检测 (Random Early Detection)
CA	拥塞算法 (Congestion Algorithm)
PPO	近端策略优化 (Proximal Policy Optimization)
DQN	深度 Q 网络 (Deep Q-Network)
PCC	面向性能的拥塞控制 (Performance-oriented Congestion Control)
ZMQ	ZeroMQ 消息队列
SDN	软件定义网络 (Software Defined Networking)

QoS	服务质量 (Quality of Service)
QUIC	快速 UDP 互联网连接 (Quick UDP Internet Connections)
RDMA	远程直接内存访问 (Remote Direct Memory Access)
PFC	优先级流量控制 (Priority Flow Control)
INT	带内网络遥测 (In-band Network Telemetry)
HPCC	高精度拥塞控制 (High Precision Congestion Control)
WAN	广域网 (Wide Area Network)
Pod	交换机组 (Point of Delivery)
EMA	指数移动平均 (Exponential Moving Average)
IPC	进程间通信 (Inter-Process Communication)
MTP	多线程并行 (Multi-Thread Parallel)

第一章 绪论

1.1 研究背景与动机

随着云计算、大数据、人工智能和物联网技术的蓬勃发展，全球数据中心网络的规模和复杂度呈现指数级增长态势。根据国际数据公司（International Data Corporation, IDC）的统计数据，全球数据中心网络流量正以年均 25% 以上的速度持续增长，预计到 2026 年将突破 25ZB（泽字节）的惊人规模^[1]。传输控制协议（Transmission Control Protocol, TCP）作为互联网传输层的核心协议，承载了超过 90% 的数据中心网络流量，其拥塞控制机制的性能直接影响着网络资源利用效率、应用响应延迟和用户服务质量。网络拥塞控制是计算机网络领域最具挑战性的研究课题之一，自互联网诞生以来一直是学术界和工业界持续关注的焦点问题。

在现代数据中心网络架构中，网络带宽经历了从千兆级别到 10Gbps、25Gbps 乃至 100Gbps 级别的跨越式提升，而网络延迟则从毫秒级降低到微秒级。以典型的 Leaf-Spine 数据中心网络架构为例，机架内（Intra-Rack）通信延迟通常在微秒级别，而跨 Pod（Inter-Pod）通信延迟也仅在数十微秒量级。这种高带宽、低延迟的网络环境对传统拥塞控制算法提出了前所未有的严峻挑战。

现代数据中心承载着多种类型的应用工作负载，这些应用对网络性能有着截然不同的需求。分布式存储系统（如 HDFS、Ceph）需要高吞吐量以支持大规模数据读写；在线事务处理系统（OLTP）和分布式数据库（如 Spanner、CockroachDB）对延迟极为敏感，通常要求 99 百分位延迟（P99 Latency）控制在毫秒级甚至亚毫秒级；机器学习训练任务（如分布式 TensorFlow、PyTorch）则对带宽利用率和多流公平性有较高要求。单一的拥塞控制策略难以同时满足这些多样化的性能需求。

更为复杂的是，数据中心网络的流量模式具有高度动态性和突发性特征。研究表明^[2]，数据中心网络流量呈现出显著的“Mice-Elephant”（小流量与大流量并存）分布特征：绝大多数流量是持续时间短、数据量小的“Mice Flow”（小鼠流），而少数持续时间长、数据量大的“Elephant Flow”（大象流）则贡献了绝大部分的网络带宽消耗。Incast（多对一）通信模式在分布式系统中十分常见，会导致瞬时的严重拥塞；而 Partition/Aggregate（分区聚合）工作负载则对尾延迟（Tail Latency）极为敏感。

传统的 TCP 拥塞控制算法在这种复杂的网络环境中显露出诸多不足。基于丢包的算法（如 TCP NewReno^[3]、TCP CUBIC^[4]）将数据包丢失作为拥塞信号，但丢包是拥塞的滞后指示器，通常意味着网络缓冲区已经溢出，排队延迟已经积累到较高水平。基于延迟的算法（如 TCP Vegas^[5]、TCP BBR^[6]）尝试通过监测 RTT 变化提前感知拥塞，但 RTT 测量易受多种因素干扰，且在与基于丢包算法共存时存在公平性问题。

显式拥塞通知（Explicit Congestion Notification, ECN）机制为解决上述问题提供了新的思路。ECN 允许网络设备在队列长度超过阈值时标记数据包而非丢弃，从而实现更早、更精确的拥塞信号传递。数据中心 TCP（DCTCP^[7]）利用 ECN 标记比例进行细粒度调整，取得了良好效果。然而，现有算法对 ECN 信号的利用仍不够充分，大多采用静态参数配置，缺乏对动态网络环境的自适应能力。

近年来，机器学习特别是强化学习技术的快速发展为网络拥塞控制带来了革命性的新思路^[8]。强化学习（Reinforcement Learning, RL）是一种通过与环境交互来学习最优策略的机器学习方法。在强化学习框架中，智能体（Agent）通过观察环境状态、执行动作并获得奖励来不断优化决策策略。将强化学习应用于网络拥塞控制，理论上能够实现：（1）自适应地学习网络环境特征，无需人工设计复杂的控制规则；（2）综合利用多种拥塞信号进行决策；（3）根据网络条件动态调整控制参数；（4）良好的可扩展性和通用性。

然而，现有基于强化学习的拥塞控制方案仍存在诸多问题：状态空间设计不完善，通常只使用 4-6 维的有限状态参数；奖励函数设计不合理，难以平衡多个优化目标；缺乏对 ECN 信号的分级利用；算法验证效率低下等。这些问题限制了强化学习在拥塞控制领域的实际应用效果。

1.2 研究目标与主要贡献

针对上述问题，本文提出了 Lark 算法——一种基于强化学习的多信号融合自适应网络拥塞控制方法。“Lark”意为“云雀”，云雀以其轻盈敏捷的飞行姿态和清脆悦耳的歌声著称。选用“Lark”作为算法名称，寓意本算法追求的设计目标：如同云雀般轻盈敏捷，实现高吞吐量（High Throughput）与低延迟（Low Latency）的完美平衡；如同云雀对环境变化的敏锐感知，实现对多种拥塞信号的快速响应和智能融合。

图1.1展示了 Lark 算法的核心贡献概览及各组件之间的逻辑关系。

本文的主要研究贡献如下：

（1）完整的 15 维观测空间设计

本文设计了一个包含 15 个参数的完整观测空间，首次将显式拥塞通知（ECN）状态、拥塞控制状态机状态、拥塞事件类型等丰富的 TCP 协议栈信息纳入强化学习状态表示。与现有方案通常采用的 4-6 维简化状态空间相比，Lark 的 15 维观测空间能够为强化学习智能体提供更全面、更精细的网络状态感知能力，为精准的拥塞控制决策奠定基础。

观测空间的 15 个参数涵盖六大类别：连接标识信息（套接字 UUID、节点 ID）用于多流识别和状态隔离；时间信息（仿真时间戳）用于时序分析和事件率计算；窗口状态参数（慢启动阈值 $ssthresh$ 、拥塞窗口 $cwnd$ ）是拥塞控制的核心状态变量；传输性能指标（段大小、确认段数、在途字节数）用于吞吐量估算和管道利用率计算；延迟测量数据（最近 RTT、最小

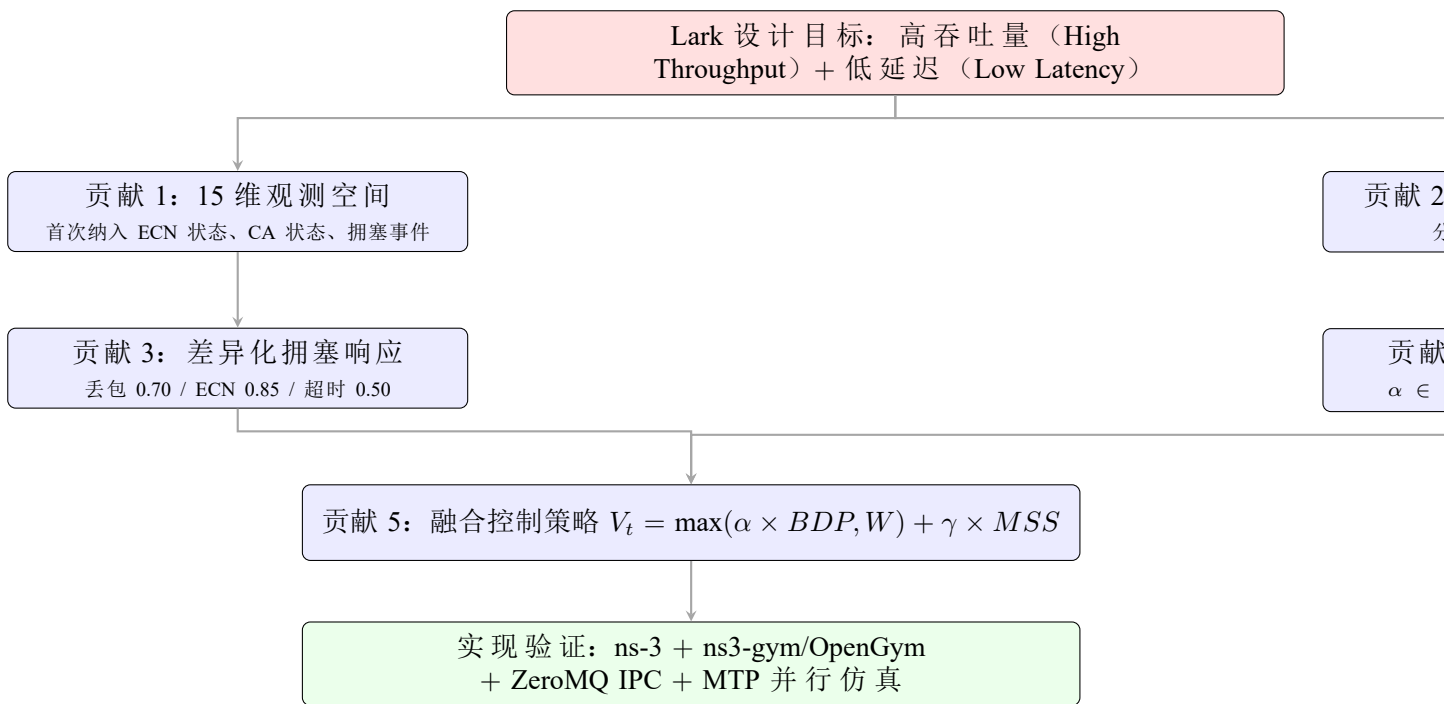


图 1.1 Lark 算法核心贡献概览

RTT) 用于带宽延迟积估算和拥塞程度判断; 回调类型标识区分丢包事件和正常确认事件; 拥塞控制状态 (CA 状态、CA 事件、ECN 状态) 提供协议栈的完整状态信息。

(2) 多信号融合拥塞检测方法

本文提出了一种多信号融合的拥塞检测方法, 采用分层优先级机制综合分析多种拥塞信号。该方法的核心思想是: 不同类型的拥塞信号具有不同的可靠性和紧迫程度, 应当采用差异化的处理策略。具体而言, 显式丢包信号具有最高优先级, 表明网络已经发生严重拥塞; 当拥塞控制状态机处于 CA_LOSS 状态时进一步区分超时丢包事件; ECN 拥塞经历 (Congestion Experienced, CE) 标记和 ECN 回显 (ECN-Echo, ECE) 接收作为次级信号, 表明网络设备已检测到队列积压。

为了避免对瞬态扰动的过度响应, 本文在拥塞检测中对不同信号类型采用差异化的触发逻辑, 确保拥塞响应的精准性和鲁棒性。这种设计在保证对真实拥塞快速响应的同时, 避免了因瞬态信号导致的不必要窗口缩减, 有效平衡了网络稳定性和吞吐量。

(3) 差异化拥塞响应机制

本文设计了差异化的拥塞响应机制, 针对不同类型的拥塞信号采用不同的窗口保留因子 (Window Retention Factor)。具体而言, 丢包触发的拥塞响应采用 0.70 的窗口保留因子, 这是最激进的响应策略, 因为丢包是最严重的拥塞信号; ECN 触发的拥塞响应采用 0.85 的窗口保留因子, 保留了较大比例的窗口以维持吞吐量, 因为 ECN 是早期拥塞信号, 网络可能只是轻度拥塞; 超时触发的拥塞响应采用 0.50 的窗口保留因子, 这是最严厉的响应策略, 因为超时通常意味着连续多个数据包丢失或网络路径严重中断。

此外，本文引入了连续缩减保护机制（Consecutive Decrease Protection）：当连续拥塞响应次数超过 3 次时，保持当前窗口不再缩减，避免因连续拥塞事件导致窗口过度收缩。这种差异化响应机制的设计理念是：拥塞信号的性质和严重程度不同，响应策略也应当有所区别。传统算法对所有拥塞信号采用统一的响应策略（如窗口减半），无法区分轻度拥塞和严重拥塞，导致在轻度拥塞时响应过度、在严重拥塞时响应不足。Lark 的差异化响应机制能够更精准地匹配拥塞程度，在保证网络稳定性的同时最大化吞吐量。

（4）强化学习驱动的自适应参数调优

本文实现了强化学习驱动的自适应参数调优机制，根据多种因素动态调整乘性增加因子（Multiplicative Increase Factor, α ）。调整因素包括：RTT 膨胀比（当前 RTT 与最小 RTT 的比值）反映网络排队延迟程度，采用三级梯度响应——当 RTT 膨胀比小于 1.3 时增大 α （步长 +0.03），介于 1.3 至 2.0 之间时温和增大 α （步长 +0.01），大于 3.0 时减小 α （步长 -0.02）；奖励信号的指数移动平均值（EMA）反映整体控制效果的趋势，正向趋势增大 α ，负向趋势减小 α ；连续增长次数超过 5 次时给予稳定增长奖励。

α 的取值被限制在 [1.05, 1.50] 范围内，基准初始值为 1.20。相比固定参数设计，这种自适应机制能够使 Lark 在不同网络环境下自动调整控制策略的激进程度：在低拥塞环境中快速提升 α 以充分利用带宽，在高拥塞环境中降低 α 以保持网络稳定。

（5）融合控制策略

本文采用融合控制策略计算目标拥塞窗口，将基于带宽延迟积（Bandwidth-Delay Product, BDP）估计的速率控制与基于当前窗口的保守控制相结合。BDP 通过窗口化最大值过滤（Windowed Max-filter）方法估计瓶颈带宽（取最近 20 个投递率样本的最大值），乘以最小 RTT 得到。在慢启动阶段，目标窗口设为 BDP 的 2 倍，采用指数增长策略以快速探测可用带宽；在拥塞避免阶段，目标窗口按公式 $V_t = \max(\alpha \times BDP, W) + \gamma \times MSS$ 计算，其中 $\gamma = 2.0$ 为加性增加因子，每个 ACK 事件增加 γ 个 MSS 的窗口增量。

融合控制策略还引入了管道利用率（Pipeline Utilization）感知机制：当管道利用率低于 70% 时，额外增加 1 个 MSS 的窗口增量以提高带宽利用效率；当管道利用率低于 40% 时，进一步增加 2 个 MSS 的窗口增量以快速恢复带宽利用。这种设计确保了 Lark 算法能够快速适应网络带宽变化，在带宽增加时迅速提升吞吐量，在带宽减少时及时收敛。

（6）每流独立状态管理

本文设计了每流独立的状态管理机制，为每个 TCP 连接维护独立的状态数据结构。状态数据包括：带宽估计数据（窗口化最大值过滤的投递率样本队列，容量 20 个样本点）、RTT 跟踪数据（最近 RTT 样本队列，容量 40 个样本点）、BDP 估计值、自适应参数 α 、拥塞计数器（连续缩减次数、连续增长次数、丢包计数、ECN 计数）、吞吐量 EMA 等。

每流独立状态管理基于哈希表实现 $O(1)$ 时间复杂度的状态查找与更新。其意义在于：在

多流并发场景下，不同流可能经历不同的网络路径和拥塞状况，采用全局统一的状态管理会导致流间干扰；而每流独立的状态管理能够实现差异化控制，提高多流场景下的整体性能和公平性。

（7）多线程并行化仿真架构

本文采用多线程并行化仿真架构加速算法训练和验证。传统的 ns-3 网络仿真器采用单线程串行架构，在大规模网络拓扑和长时间仿真场景下效率较低。本文采用 MTP-Interface (Multi-Thread Parallel Interface) 多线程接口，将网络拓扑划分为多个逻辑分区，在多个线程上并行执行各分区的仿真事件，通过保守同步协议协调线程间的事件同步。实验表明，在 8 核 CPU 上可获得 3-5 倍的仿真加速。

1.3 论文组织结构

本文的其余部分组织如下：

第 2 章介绍相关工作，包括传统拥塞控制算法、基于机器学习的拥塞控制方法、显式拥塞通知机制、强化学习在网络领域的应用等。

第 3 章详细描述 Lark 算法的设计与实现，包括系统架构、观测空间设计、多信号融合拥塞检测、差异化拥塞响应、自适应参数调优、融合控制策略等核心组件。

第 4 章介绍实验评估，包括实验环境配置、评测指标、对比算法、实验场景设计以及详细的实验结果分析。

第 5 章讨论 Lark 算法的局限性和未来研究方向。

第 6 章总结全文并展望未来工作。

第二章 相关工作

网络拥塞控制是计算机网络领域的核心研究课题之一，自互联网诞生以来一直是学术界和工业界关注的焦点。本章从传统拥塞控制算法、显式拥塞通知机制、基于机器学习的拥塞控制方法三个方面综述相关研究工作，并分析各方法的优缺点及其对 Lark 算法设计的启示。

图2.1展示了拥塞控制算法的分类体系及其演进关系。

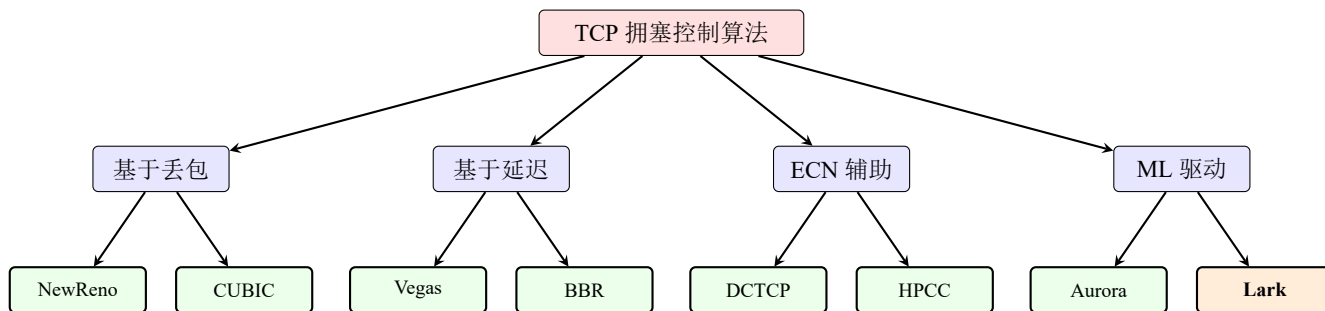


图 2.1 拥塞控制算法分类体系

2.1 传统拥塞控制算法

传统的 TCP 拥塞控制算法根据其使用的拥塞信号类型，可以分为基于丢包的算法、基于延迟的算法和混合算法三大类。

2.1.1 基于丢包的拥塞控制算法

基于丢包的拥塞控制算法将数据包丢失作为网络拥塞的主要判断信号。这类算法的核心思想是：当发送端检测到数据包丢失（通过超时或重复确认判断）时，认为网络发生了拥塞，需要降低发送速率以缓解拥塞。

TCP Tahoe 和 **TCP Reno** 是最早的拥塞控制算法，由 Van Jacobson 于 1988 年提出^[9]。TCP Tahoe 引入了慢启动（Slow Start）、拥塞避免（Congestion Avoidance）和快速重传（Fast Retransmit）三个核心机制。慢启动机制在连接建立初期指数级增加拥塞窗口，快速探测可用带宽；拥塞避免机制在窗口达到阈值后线性增加窗口，谨慎探测额外带宽；快速重传机制在收到三个重复确认时立即重传丢失的数据包，而无需等待超时。TCP Reno 在 Tahoe 的基础上增加了快速恢复（Fast Recovery）机制，在快速重传后将窗口减半而非重置为 1，提高了丢包后的恢复效率。

TCP NewReno^[3] 是对 TCP Reno 的改进，主要解决了 Reno 在多个数据包丢失时性能下降的问题。NewReno 修改了快速恢复机制的行为：在快速恢复期间，如果收到部分确认（Partial

ACK, 确认了部分但非全部已发送数据), 则继续保持在快速恢复状态, 直到所有丢失的数据包都被成功重传。这种改进使得 NewReno 能够更高效地处理突发丢包情况, 成为 RFC 6582 中定义的标准 TCP 拥塞控制算法之一。

TCP CUBIC^[4] 是目前 Linux 操作系统的默认拥塞控制算法, 自 Linux 2.6.19 版本开始采用。CUBIC 的核心创新是采用三次函数 (Cubic Function) 模型调整拥塞窗口大小:

$$W(t) = C \cdot (t - K)^3 + W_{max} \quad (2.1)$$

其中, $W(t)$ 是时刻 t 的窗口大小, C 是缩放因子, $K = \sqrt[3]{\frac{W_{max} \cdot \beta}{C}}$ 是从上次拥塞事件恢复到 W_{max} 所需的时间, W_{max} 是上次拥塞事件前的窗口大小, β 是窗口缩减因子。

CUBIC 的三次函数模型具有以下特点: (1) 在远离 W_{max} 时快速增长, 在接近 W_{max} 时缓慢增长, 实现了对可用带宽的稳定探测; (2) 窗口增长速度与 RTT 无关, 避免了传统 AIMD 算法在高 RTT 链路上收敛缓慢的问题; (3) 在 W_{max} 附近有一个“平台期”, 减少了在最优工作点附近的窗口振荡。

TCP BIC^[10] 是 CUBIC 的前身, 采用二分搜索 (Binary Search) 算法调整拥塞窗口。BIC 的基本思想是: 上次拥塞发生时的窗口 W_{max} 是带宽上限的估计, 当前窗口是带宽下限的估计, 通过二分搜索在两者之间寻找最优窗口。BIC 算法的窗口增长函数如下:

$$W_{new} = W_{current} + \frac{W_{max} - W_{current}}{2} \quad (2.2)$$

当窗口接近 W_{max} 时, BIC 采用加性增加策略以避免过冲。BIC 算法在高带宽延迟积网络中表现良好, 但其窗口增长行为对 RTT 不敏感, 在与其他 TCP 变体共存时可能产生公平性问题。

基于丢包的算法存在以下共同缺陷:

(1) **拥塞信号滞后**: 丢包是网络拥塞的滞后指示器。网络设备的缓冲区在溢出丢包前已经积累了大量排队数据, 导致排队延迟显著增加。在深缓冲区 (Deep Buffer) 网络环境中, 这种延迟可能达到数十甚至数百毫秒级别, 严重影响延迟敏感型应用的性能。

(2) **缓冲区膨胀**: 基于丢包算法的典型行为是持续增加发送速率直到发生丢包, 这会导致网络缓冲区长期处于高占用状态, 产生“Bufferbloat” (缓冲区膨胀) 问题^[11]。缓冲区膨胀不仅增加了网络延迟, 还降低了网络的整体吞吐量, 因为大量带宽被用于传输最终会被丢弃的数据包。

(3) **无法区分拥塞程度**: 基于丢包算法只能检测“有丢包”或“无丢包”两种状态, 无法区分轻度拥塞和严重拥塞。因此, 无论拥塞程度如何, 算法都采用统一的窗口缩减策略, 在轻度拥塞时可能响应过度, 在严重拥塞时又可能响应不足。

(4) **高 BDP 网络收敛缓慢**: 在高带宽延迟积网络中, 基于丢包算法的收敛速度极为缓慢。以 10Gbps 带宽、100 微秒 RTT 的链路为例, BDP 约为 125KB, 需要约 86 个 MSS 的窗口才能充分利用带宽。从初始窗口增长到 86 个 MSS 需要较长时间, 丢包后的恢复更是如此。

2.1.2 基于延迟的拥塞控制算法

基于延迟的拥塞控制算法通过监测往返时延 (RTT) 的变化来推断网络拥塞状态, 试图在丢包发生前就感知到拥塞并采取措施。这类算法的核心假设是: 网络拥塞会导致队列积累, 进而导致 RTT 增加。

TCP Vegas^[5] 是最早的基于延迟拥塞控制算法之一, 由 Lawrence Brakmo 和 Larry Peterson 于 1994 年提出。Vegas 通过比较期望吞吐量和实际吞吐量来估计网络中的排队数据量。期望吞吐量定义为:

$$Expected = \frac{cwnd}{BaseRTT} \quad (2.3)$$

实际吞吐量定义为:

$$Actual = \frac{cwnd}{RTT} \quad (2.4)$$

其中, $BaseRTT$ 是测量到的最小 RTT, 被认为是无排队情况下的传播延迟。差值 $\Delta = Expected - Actual$ 表示网络中排队的的数据量。Vegas 根据 Δ 的大小调整拥塞窗口:

$$cwnd = \begin{cases} cwnd + 1 & \text{如果 } \Delta < \alpha \\ cwnd - 1 & \text{如果 } \Delta > \beta \\ cwnd & \text{否则} \end{cases} \quad (2.5)$$

其中, α 和 β 是预设的阈值参数, 通常设置为 $\alpha = 1$ 、 $\beta = 3$ 。

TCP BBR^[6] (Bottleneck Bandwidth and Round-trip propagation time) 是 Google 于 2016 年提出的基于延迟算法, 是近年来最具影响力的拥塞控制算法之一。BBR 的核心思想是通过显式估计瓶颈带宽 (Bottleneck Bandwidth, BtlBw) 和最小 RTT (RTT_{prop}), 以带宽延迟积为目标调整发送速率。

BBR 通过维护一个滑动窗口来跟踪最大投递率 (作为 BtlBw 的估计) 和最小 RTT (作为 RTT_{prop} 的估计)。发送速率和拥塞窗口按以下方式计算:

$$pacing_rate = pacing_gain \times BtlBw \quad (2.6)$$

$$cwnd = cwnd_gain \times BDP = cwnd_gain \times BtlBw \times RTT_{prop} \quad (2.7)$$

BBR 采用周期性的增益调整策略，在不同阶段采用不同的 $pacing_gain$ 和 $cwnd_gain$ 值，以实现带宽探测和 RTT 测量。

TCP Copa^[12] 是 MIT 于 2018 年提出的基于延迟算法，采用了与 Vegas 类似但更精细的 RTT 监测机制。Copa 将目标排队延迟设定为 $RTT_{standing} - RTT_{min}$ 的函数，其中 $RTT_{standing}$ 是最近一个 RTT 窗口内的最小 RTT。Copa 采用以下窗口调整规则：

$$cwnd = cwnd + \frac{1}{cwnd \cdot \delta} \quad \text{或} \quad cwnd = cwnd - \frac{1}{cwnd \cdot \delta} \quad (2.8)$$

其中， δ 是目标延迟参数，默认值为 0.5。Copa 还引入了“竞争模式”检测机制，当检测到与基于丢包算法的流竞争时，会切换到更激进的发送策略。

基于延迟的算法存在以下共同问题：

(1) **RTT 测量不稳定**：RTT 测量受多种因素影响，包括操作系统调度延迟、网络路径抖动、网络拓扑变化、对端主机处理延迟等。在高速网络中，这些干扰因素可能与实际排队延迟处于同一数量级，导致基于 RTT 的拥塞推断不可靠。

(2) **公平性问题**：基于延迟算法在与基于丢包算法共存时存在严重的公平性问题。由于基于延迟算法对拥塞信号更敏感，会在 RTT 增加时提前降低发送速率，而基于丢包算法继续增加窗口直到丢包。研究表明，在与 CUBIC 流共存时，BBR 流的吞吐量可能下降 30% 以上^[13]。

(3) **参数敏感性**：基于延迟算法的性能对参数设置较为敏感。例如，Vegas 的 α 和 β 参数、Copa 的 δ 参数都需要根据具体网络环境进行调优，而静态参数难以适应动态变化的网络条件。

2.1.3 混合拥塞控制算法

混合拥塞控制算法综合利用丢包和延迟两种信号进行拥塞控制，试图结合两类算法的优点。

Compound TCP^[14] 是 Microsoft 开发的混合算法，在 Windows Vista 及后续版本中使用。Compound TCP 在传统的基于丢包的窗口增长之上叠加了一个基于延迟的“快速”窗口分量，在网络空闲时快速增长，在队列积累时减速增长。

TCP Illinois^[15] 根据 RTT 变化调整 AIMD 参数，在低延迟时采用更激进的增加策略，在高延迟时采用更保守的增加策略和更激进的减少策略。

TCP Westwood^[16] 通过测量 ACK 到达率来估计可用带宽，在丢包时根据估计的带宽而非固定比例调整慢启动阈值。

混合算法在一定程度上缓解了单一信号类型的局限性，但也增加了算法复杂性，且参数调优更加困难。

2.2 显式拥塞通知机制

显式拥塞通知（Explicit Congestion Notification, ECN）是由 IETF 在 RFC 3168^[17] 中定义的一种网络层辅助拥塞控制机制。ECN 通过修改 IP 报头中的服务类型字段（Type of Service, ToS）来传递拥塞信息，实现了比隐式丢包信号更早、更精确的拥塞通知。

2.2.1 ECN 工作原理

ECN 在 IP 报头中使用两个比特（ECN 字段），定义了四种码点：

- 00: Non-ECT（非 ECN 能力传输），表示端点不支持 ECN
- 01: ECT(1)（ECN 能力传输 1），表示端点支持 ECN
- 10: ECT(0)（ECN 能力传输 0），表示端点支持 ECN
- 11: CE（Congestion Experienced，拥塞经历），由路由器标记，表示经历了拥塞

ECN 的工作流程如下：发送端在发送数据包时将 ECN 字段设置为 ECT(0) 或 ECT(1)，表明该数据包支持 ECN。当网络设备（如路由器）检测到队列长度超过预设阈值时，将 ECN 字段从 ECT(0/1) 修改为 CE，而不是丢弃数据包。接收端收到 CE 标记的数据包后，在 TCP 报头中设置 ECE（ECN-Echo）标志，将拥塞信息反馈给发送端。发送端收到 ECE 标志后，采取拥塞响应措施（如降低发送速率），并在随后的数据包中设置 CWR（Congestion Window Reduced）标志，通知接收端已经响应了拥塞通知。图2.2展示了 ECN 信号传递的完整工作流程。

2.2.2 ECN 相关拥塞控制算法

DCTCP^[7]（Data Center TCP）是 Microsoft 和 Stanford 于 2010 年提出的数据中心专用拥塞控制算法，是 ECN 应用的典范。DCTCP 的核心创新是利用 ECN 标记的比例（而非仅仅是有无）来进行细粒度的窗口调整。DCTCP 维护一个指数加权移动平均（EWMA）的 ECN 标记比例 α ：

$$\alpha = (1 - g) \cdot \alpha + g \cdot F \quad (2.9)$$

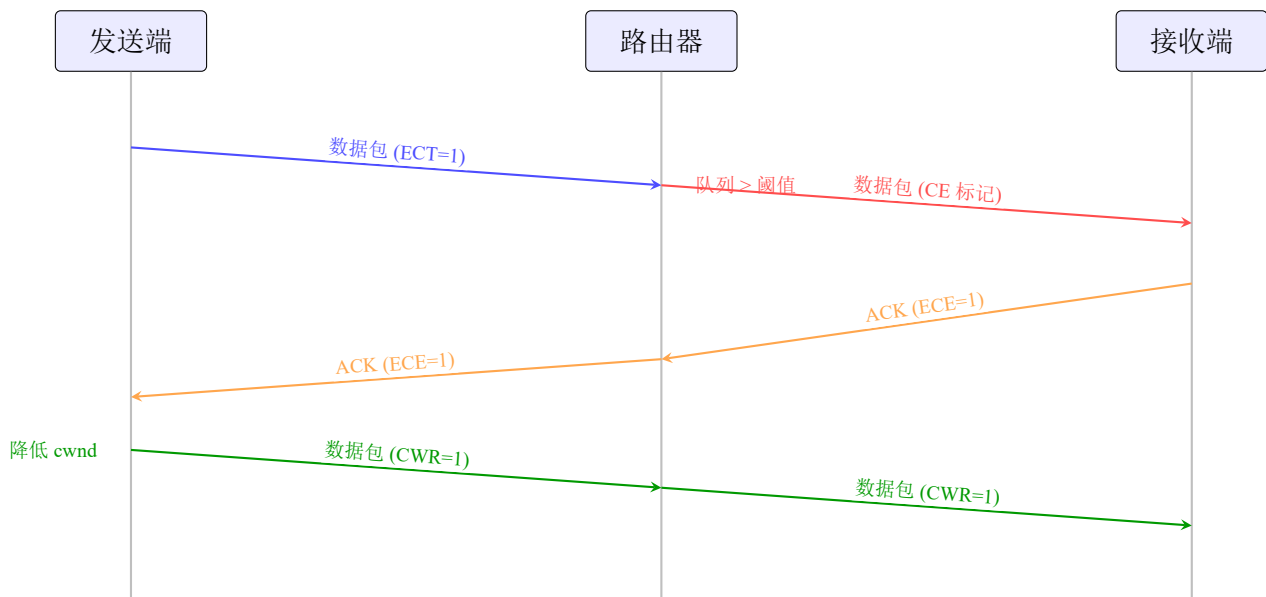


图 2.2 ECN 信号传递工作流程

其中, g 是平滑因子 (通常为 $1/16$), F 是最近一个 RTT 内被标记的数据包比例。拥塞窗口根据 α 进行调整:

$$cwnd = cwnd \times \left(1 - \frac{\alpha}{2}\right) \quad (2.10)$$

这种设计使得 DCTCP 能够根据拥塞程度进行比例响应: 轻度拥塞时 (α 较小) 轻微降低窗口, 严重拥塞时 (α 较大) 大幅降低窗口。

DCTCP 要求网络交换机配置适当的 ECN 标记阈值, 通常使用 RED (Random Early Detection) 或其变体。推荐的配置是将标记阈值设置为较低的值 (如 20 个数据包或 BDP 的 10%), 以实现早期拥塞通知。

ECN++^[18] 是对 DCTCP 的改进, 引入了更精细的 ECN 标记策略和响应机制。ECN++ 在接收端累积 ECN 标记计数, 而非仅使用二元 ECE 标志, 从而传递更丰富的拥塞信息。

HPCC^[19] (High Precision Congestion Control) 是阿里巴巴于 2019 年提出的高精度拥塞控制算法, 利用网络设备提供的精确拥塞信息 (如队列长度、带宽利用率) 进行拥塞控制。HPCC 要求网络设备支持 INT (In-band Network Telemetry) 功能。

ECN 机制的主要优势在于: (1) 能够在丢包发生前提供明确的拥塞信号; (2) 避免了因丢包导致的重传开销; (3) 信号来自网络设备的显式标记, 比推测性的 RTT 测量更可靠。然而, ECN 的有效使用需要网络设备的支持和正确配置, 且现有算法对 ECN 信号的利用还不够充分。

2.3 基于机器学习的拥塞控制方法

近年来，机器学习技术在网络拥塞控制领域的应用受到了广泛关注。根据所采用的学习范式，可以分为监督学习方法、强化学习方法和其他方法。

2.3.1 监督学习方法

监督学习方法通过从标记数据中学习拥塞控制策略。

Remy^[20] 是 MIT 于 2013 年提出的开创性工作，采用离线优化方法生成拥塞控制规则。Remy 将拥塞控制问题形式化为一个优化问题，在给定网络模型的情况下，搜索最优的窗口调整规则。Remy 的规则形式为：在特定状态下（由 RTT 测量、ACK 间隔等特征定义），执行特定的窗口增量和发送间隔调整。

Remy 的优点是能够针对特定网络环境生成最优规则，在目标场景下表现优异。其局限性在于：（1）需要准确的网络模型，对模型误差敏感；（2）离线生成的规则难以适应动态变化的网络环境；（3）状态空间划分的粒度影响算法性能和复杂度。

2.3.2 强化学习方法

强化学习方法通过与网络环境的交互来学习最优的拥塞控制策略，能够适应动态变化的网络条件。

Aurora^[21] 是 CMU 于 2019 年提出的基于深度强化学习的拥塞控制算法。Aurora 采用 PPO（Proximal Policy Optimization）算法训练神经网络策略，输入网络状态特征（延迟梯度、延迟比率、发送比率等），输出发送速率调整因子。Aurora 的状态空间包含 10 个特征，涵盖延迟、吞吐量和丢包率的历史统计信息。

Aurora 的奖励函数设计为吞吐量和延迟的加权组合：

$$reward = throughput - \delta \cdot delay - \lambda \cdot loss \quad (2.11)$$

其中， δ 和 λ 是权重参数，控制延迟和丢包的惩罚程度。

Orca^[22] 是 UC Berkeley 于 2020 年提出的强化学习拥塞控制算法，采用经典的 AIMD 框架，但使用强化学习来动态调整 AIMD 参数。Orca 的核心思想是：AI 参数（增加因子 α 和减少因子 β ）不应是固定值，而应根据网络状态动态调整。

PCC^[23]（Performance-oriented Congestion Control）是 UIUC 提出的基于在线学习的拥塞控制算法。PCC 采用实用函数（Utility Function）指导发送速率调整，通过在线实验探索最优速率。

$$U(x) = T \cdot x^\alpha - \delta \cdot L \cdot x - \beta \cdot D \cdot x \quad (2.12)$$

其中, x 是发送速率, T 是吞吐量, L 是丢包率, D 是延迟, α 、 δ 、 β 是参数。PCC 通过梯度上升方法最大化实用函数。

现有基于强化学习的方法存在以下共同问题:

- (1) **状态空间设计不完善:** 现有方案通常只使用 4-6 维的简化状态参数, 未能充分利用 TCP 协议栈提供的丰富状态信息, 特别是 ECN 相关状态和拥塞控制事件信息。
- (2) **奖励函数设计困难:** 吞吐量、延迟、公平性、丢包率等多个优化目标之间存在权衡关系, 设计能够平衡多目标的奖励函数是一个挑战。
- (3) **训练效率低下:** 网络仿真的计算开销较大, 限制了强化学习算法的训练效率。
- (4) **泛化能力有限:** 在特定网络环境下训练的策略可能难以泛化到其他环境。

2.4 数据中心网络特性分析

为了更好地理解 Lark 算法的设计动机, 本节深入分析现代数据中心网络的特性。

2.4.1 网络拓扑特性

现代数据中心网络普遍采用 Leaf-Spine (叶脊) 或 Fat-Tree (胖树) 拓扑结构, 取代了传统的三层树形结构。这种新型拓扑具有以下特点:

- (1) **多路径冗余:** 任意两个服务器之间存在多条等价路径, 提供了负载均衡和故障恢复能力。典型的 Leaf-Spine 拓扑中, 一对叶子交换机之间的等价路径数量等于脊交换机数量。
- (2) **低直径网络:** Leaf-Spine 拓扑的网络直径 (任意两个节点之间的最大跳数) 通常为 4-6 跳, 远小于传统三层网络的 10 跳以上。较低的网络直径意味着较低的端到端延迟。
- (3) **高带宽密度:** 现代数据中心交换机支持高密度的高速端口, 单台交换机可提供数百 Tbps 的总交换容量。网络带宽资源相对充裕, 但在流量聚合点 (如汇聚层、核心层) 仍可能出现拥塞。
- (4) **均匀延迟:** 由于网络直径较低且拓扑规则, 数据中心内部的端到端延迟相对均匀, 通常在微秒级别。这使得基于延迟的拥塞检测成为可能, 但也对算法的灵敏度提出了更高要求。

2.4.2 流量特性分析

数据中心网络的流量特性与广域网有显著差异:

(1) **Mice-Elephant 分布**: 数据中心流量呈现明显的双峰分布, 绝大多数流量是持续时间短 (小于 10ms)、数据量小 (小于 10KB) 的”小鼠流”(Mice Flow), 而少数持续时间长、数据量大的”大象流”(Elephant Flow) 贡献了绝大部分的带宽消耗。研究表明, 超过 80% 的流量是小于 10KB 的短流, 但超过 80% 的带宽被不到 1% 的大流消耗。

(2) **Incast 通信模式**: 分布式系统中普遍存在”多对一”的 Incast 通信模式, 例如分布式存储系统中多个节点向协调节点发送数据、MapReduce 的 Shuffle 阶段、分布式数据库的查询聚合等。Incast 会导致接收端瞬间收到大量数据, 造成交换机缓冲区溢出和严重的数据包丢失。

(3) **流量突发性**: 数据中心应用产生的流量具有高度突发性, 突发流量可能在微秒级时间尺度上出现, 远快于传统拥塞控制算法的响应速度。

(4) **流量的时间局部性**: 数据中心应用通常表现出较强的时间局部性, 即短时间内重复访问相同的数据或服务。这种特性可以被缓存和预取机制利用。

2.4.3 应用延迟需求分析

不同类型的数据中心应用对网络延迟有不同的需求:

(1) **在线交易处理 (OLTP)**: 电子商务、金融交易等应用对延迟极为敏感, 通常要求 99 百分位延迟 (P99) 控制在毫秒级甚至亚毫秒级。研究表明, 延迟每增加 100ms, 电商网站的转化率下降约 7%。

(2) **实时分析 (OLAP)**: 商业智能、实时报表等应用要求快速返回分析结果, 通常可接受的延迟在秒级到分钟级。但用户对延迟的感知会影响使用体验。

(3) **批量处理**: 数据挖掘、机器学习训练等应用对延迟不敏感, 更关注吞吐量和总完成时间。这类应用可以容忍较高的网络延迟, 但对带宽利用效率有较高要求。

(4) **实时通信**: 视频会议、协作办公等应用要求持续的低延迟, 通常端到端延迟应控制在 150ms 以内以保证良好的用户体验。

Lark 算法的”高吞吐、低延迟”设计目标正是为了同时满足多种应用类型的需求: 对于延迟敏感型应用, 提供低队列延迟; 对于吞吐量敏感型应用, 保持高带宽利用率。

2.4.4 网络设备特性分析

数据中心网络设备的特性对拥塞控制算法有重要影响:

(1) **浅缓冲区交换机**: 现代数据中心交换机通常采用浅缓冲区设计 (每端口数百 KB 到数 MB), 以降低成本和功耗。浅缓冲区意味着拥塞发生时更容易丢包, 这对拥塞控制算法的响应速度提出了更高要求。

(2) **ECN 支持**: 大多数现代数据中心交换机支持 ECN 功能, 可以在队列超过阈值时标记数据包而非丢弃。ECN 的正确配置对于基于 ECN 的拥塞控制算法至关重要。

(3) **优先级流量控制 (PFC)**: 以太网 RDMA (RoCE) 需要 PFC 支持以实现无损网络。但 PFC 可能导致“HOL 阻塞”和“拥塞扩散”等问题, 需要拥塞控制算法配合缓解。

(4) **可编程交换机**: P4 等可编程交换机技术的出现, 使得在交换机上实现自定义拥塞控制逻辑成为可能。这为未来的网络内计算拥塞控制 (In-Network Congestion Control) 开辟了新方向。

2.5 拥塞控制的理论基础

本节介绍网络拥塞控制的理论基础, 包括流量模型、排队论分析和控制论视角, 为理解 Lark 算法的设计决策提供理论支撑。

2.5.1 网络流量模型

网络流量建模是拥塞控制研究的基础。常用的流量模型包括:

(1) **泊松模型**: 传统上假设数据包到达服从泊松过程, 即到达间隔服从指数分布。泊松模型的数学性质简单, 便于分析, 但实际网络流量通常表现出更强的突发性。

(2) **自相似模型**: 研究表明, 网络流量具有自相似 (Self-Similar) 特性, 即在不同时间尺度上呈现相似的统计特性。自相似流量可以用分数布朗运动 (Fractional Brownian Motion) 或 Pareto 分布建模。设流量过程为 $X(t)$, 其自相似性定义为:

$$X(t) \stackrel{d}{=} m^{-H} X(mt), \quad \forall m > 0 \quad (2.13)$$

其中 H 是 Hurst 参数 ($0.5 < H < 1$), $\stackrel{d}{=}$ 表示同分布。

(3) **ON-OFF 模型**: TCP 流量可以用 ON-OFF 模型描述, 其中 ON 状态对应数据传输, OFF 状态对应等待。如果 ON 和 OFF 的持续时间服从重尾分布, 聚合流量将表现出自相似特性。

(4) **流体模型**: 在分析大规模网络时, 可以将离散的数据包流近似为连续的流体流。设发送速率为 $x(t)$, 队列长度为 $q(t)$, 链路容量为 C , 则流体模型为:

$$\dot{q}(t) = x(t) - C, \quad \text{当 } q(t) > 0 \text{ 或 } x(t) > C \quad (2.14)$$

Lark 算法的设计考虑了流量的突发性特征。通过 ECN 信号的频率过滤, Lark 能够区分持续性拥塞和由流量突发引起的瞬态队列积累。

2.5.2 排队论分析

排队论为分析网络缓冲区行为提供了理论工具。考虑单个瓶颈链路，可以建模为 M/M/1 队列（泊松到达、指数服务时间、单服务器）或更一般的 G/G/1 队列。

M/M/1 队列的稳态性质：设到达率为 λ ，服务率为 μ ，负载为 $\rho = \lambda/\mu$ （ $\rho < 1$ 时队列稳定），则：

平均队列长度：

$$\bar{Q} = \frac{\rho}{1 - \rho} \quad (2.15)$$

平均等待时间（Little 公式）：

$$\bar{W} = \frac{\bar{Q}}{\lambda} = \frac{1}{\mu - \lambda} \quad (2.16)$$

队列长度分布：

$$P(Q = k) = (1 - \rho)\rho^k, \quad k = 0, 1, 2, \dots \quad (2.17)$$

从排队论角度，Lark 的低延迟特性源于其维持较低的平均负载 ρ 。通过对 ECN 信号的敏感响应，Lark 在 ρ 接近 1 之前就降低发送速率，从而保持 $\bar{Q}$ 和 $\bar{W}$ 在较低水平。

ECN 标记概率：设队列长度为 Q ，ECN 标记阈值为 K ，则在 RED (Random Early Detection) 策略下：

$$P(\text{标记}|Q) = \begin{cases} 0 & Q < \text{min_th} \\ \frac{Q - \text{min_th}}{\text{max_th} - \text{min_th}} & \text{min_th} \leq Q < \text{max_th} \\ 1 & Q \geq \text{max_th} \end{cases} \quad (2.18)$$

Lark 对 ECN 的利用采用直接状态检测机制：当 `ecnState` 为 `ECN_CE_RCVD` 或 `ECN_ECE_RCVD` 时，判定存在 ECN 拥塞信号，触发保留因子为 0.85 的温和窗口缩减。这种直接检测方式避免了基于事件率的间接测量，具有更低的响应延迟。

2.5.3 控制论视角

从控制论角度，TCP 拥塞控制可以建模为反馈控制系统。系统框图包括：

- 被控对象：网络（包括链路、路由器、队列）
- 控制器：拥塞控制算法
- 反馈信号：ACK、RTT、ECN 标记、丢包
- 控制输入：发送窗口或发送速率

- 干扰：跨流量、背景噪声

线性化分析：在稳态点附近，TCP 系统可以线性化。设稳态发送速率为 x^* ，稳态队列长度为 q^* ，令 $\delta x = x - x^*$ ， $\delta q = q - q^*$ ，则线性化系统为：

$$\dot{\delta q} = \delta x \quad (2.19)$$

$$\delta x(t) = -K \cdot \delta q(t - \tau) \quad (2.20)$$

其中 K 是控制增益， τ 是反馈延迟（约等于 RTT）。

系统的特征方程为：

$$s + Ke^{-s\tau} = 0 \quad (2.21)$$

稳定性条件（小增益定理）：

$$K\tau < \frac{\pi}{2} \quad (2.22)$$

这意味着在高延迟网络中，控制增益 K 必须足够小以保证稳定性，但过小的 K 会导致收敛缓慢。Lark 的自适应参数调优机制可以理解为根据 τ （通过 RTT 测量）动态调整 K （通过 α 参数）。

时滞系统分析：网络的反馈延迟（RTT）使得拥塞控制成为时滞系统。时滞系统的稳定性分析比常微分方程系统更复杂。Lark 通过综合使用多种拥塞信号，减少了对单一信号（如 RTT）的依赖，提高了系统的鲁棒性。

2.6 强化学习在网络中的应用综述

强化学习技术在网络领域的应用日益广泛。本节综述强化学习在路由优化、资源分配、安全防护等方面的研究进展，为 Lark 算法的强化学习设计提供参考。

2.6.1 路由优化

强化学习可以用于学习最优路由策略。Q-routing 算法将每个路由器建模为智能体，学习到各目的地的最优下一跳。Deep Q-Network (DQN) 被应用于软件定义网络 (SDN) 中的流量工程，学习全局最优的路由配置。

与路由优化相比，拥塞控制的特点是：（1）时间尺度更细（毫秒级 vs. 秒级）；（2）状态空间更大（每个流独立状态）；（3）实时性要求更高。这些特点决定了 Lark 采用基于规则的决策机制，而非深度神经网络。

2.6.2 资源分配

在云计算和边缘计算中，强化学习被用于动态资源分配。例如，使用深度强化学习进行虚拟机放置、容器调度、带宽分配等。这些应用的决策周期通常在秒级到分钟级，对实时性要求较低。

Lark 算法借鉴了资源分配中的多目标优化思想，通过差异化响应机制平衡吞吐量和延迟两个目标。

2.6.3 安全防护

强化学习在网络安全领域也有应用，如入侵检测、DDoS 防护等。与安全应用不同，拥塞控制需要处理合法流量，不能简单地丢弃可疑流量，需要更精细的控制策略。

2.6.4 与 Lark 的关系

Lark 算法继承了强化学习的核心思想——通过与环境交互学习最优策略，但在具体实现上做了以下调整：

- 采用基于规则的策略而非神经网络策略，保证实时性和可解释性
- 将观测空间扩展到 15 维，充分利用 TCP 协议栈信息
- 实现每流独立状态管理，支持大规模并发场景
- 通过自适应参数调优实现策略的在线优化

表2.1从多个维度对比了上述代表性拥塞控制算法与 Lark 的技术特性。

表 2.1 代表性拥塞控制算法综合对比

特性	NewReno	CUBIC	BBR	DCTCP	Lark
拥塞信号类型	丢包	丢包	RTT	ECN 比例	多信号融合
ECN 利用	否	否	否	是	是
差异化响应	否	否	否	部分	是
参数自适应	否	否	否	否	是
BDP 估计	否	否	是	否	是
每流状态管理	是	是	是	是	是
可解释性	高	高	中	高	高
学习能力	无	无	无	无	在线自适应

2.7 本章小结

本章综述了网络拥塞控制领域的相关研究工作。传统的基于丢包算法（如 NewReno、CU-BIC）虽然成熟稳定，但存在拥塞信号滞后、缓冲区膨胀等问题；基于延迟算法（如 Vegas、BBR）能够提前感知拥塞，但 RTT 测量不稳定且公平性较差；ECN 机制提供了更精确的拥塞信号，但现有算法的利用还不够充分；基于机器学习的方法展现出自适应学习的潜力，但仍存在状态空间设计、奖励函数设计、训练效率等方面的问题。

Lark 算法在现有工作的基础上，通过设计完整的 15 维观测空间、提出多信号融合拥塞检测方法、实现差异化拥塞响应机制和自适应参数调优，试图解决上述问题，实现高吞吐量和低延迟的拥塞控制目标。

第三章 Lark 算法设计与实现

本章详细介绍 Lark 算法的设计与实现，包括系统架构概述、强化学习问题建模、15 维观测空间设计、多信号融合拥塞检测算法、差异化拥塞响应机制、强化学习驱动的自适应参数调优、融合控制策略以及每流独立状态管理机制。

3.1 系统架构概述

Lark 算法的系统架构如图3.1所示，由四个核心模块组成：网络仿真模块、拥塞控制环境模块、智能决策模块和进程间通信模块。

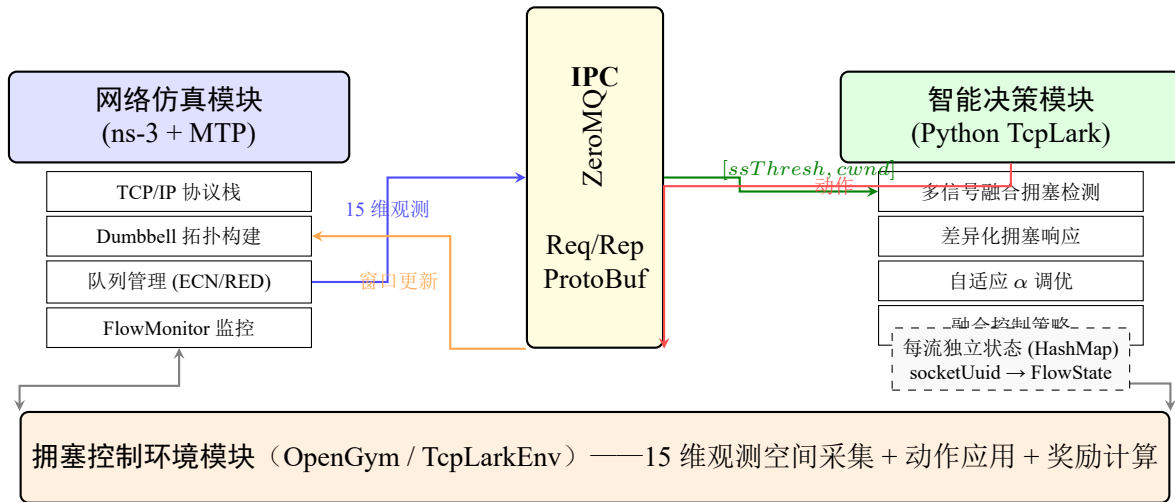


图 3.1 Lark 系统架构

网络仿真模块基于 ns-3 (Network Simulator 3) 离散事件仿真平台构建，实现了完整的 TCP/IP 协议栈。ns-3 是目前学术界和工业界广泛使用的网络仿真工具，提供了丰富的网络协议模型和灵活的可扩展架构。网络仿真模块负责构建网络拓扑（如 Leaf-Spine 拓扑、哑铃型拓扑等）、模拟数据包的传输过程、执行离散事件驱动的仿真调度。

为了提高仿真效率，网络仿真模块采用了多线程并行化架构 (MTP-Interface)。传统的 ns-3 采用单线程串行架构，在大规模网络拓扑和长时间仿真场景下效率较低。MTP-Interface 将网络拓扑划分为多个逻辑分区，在多个线程上并行执行各分区的仿真事件，通过保守同步协议 (Conservative Synchronization Protocol) 协调线程间的事件同步。

拥塞控制环境模块基于 OpenGym 框架实现，负责在 TCP 协议栈与强化学习智能体之间建立桥梁。OpenGym 是 ns3-gym 项目提供的强化学习环境接口，遵循 OpenAI Gym 的标准 API 规范。拥塞控制环境模块在 TCP 协议栈的关键回调点采集状态参数，构建 15 维观测空间，并将智能体的决策动作应用于 TCP 连接。

拥塞控制环境模块实现了以下回调接口：

- `GetSsThresh()`：慢启动阈值获取回调，在检测到丢包时调用
- `IncreaseWindow()`：窗口增加回调，在收到正常确认时调用
- `PktsAcked()`：数据包确认回调，用于 RTT 测量和吞吐量计算
- `CongestionStateSet()`：拥塞状态设置回调，用于跟踪拥塞控制状态机变化
- `CwndEvent()`：拥塞窗口事件回调，用于捕获 ECN 标记等事件

智能决策模块采用 Python 实现，负责执行 Lark 算法的核心逻辑：多信号融合拥塞检测、差异化拥塞响应、自适应参数调优和融合控制策略计算。智能决策模块维护每个 TCP 连接的独立状态，包括历史度量缓冲区、ECN 事件跟踪数据、性能指标数据和自适应参数数据。

进程间通信模块采用 ZeroMQ (ZMQ) 消息队列实现仿真进程和决策进程之间的高效通信。通信协议采用请求-应答 (Request-Reply) 模式：仿真进程在每个回调点发送观测数据请求，决策进程返回控制动作应答。消息格式采用 Protocol Buffers 序列化，保证了通信效率和跨语言兼容性。

3.2 强化学习问题建模

Lark 算法将网络拥塞控制问题建模为马尔可夫决策过程 (Markov Decision Process, MDP)。MDP 是强化学习的标准数学框架，由五元组 $\langle S, A, P, R, \gamma \rangle$ 定义：

- S ：状态空间，表示所有可能的网络状态
- A ：动作空间，表示所有可能的控制动作
- P ：状态转移概率函数， $P(s'|s, a)$ 表示在状态 s 执行动作 a 后转移到状态 s' 的概率
- R ：奖励函数， $R(s, a, s')$ 表示状态转移的即时奖励
- γ ：折扣因子， $0 \leq \gamma \leq 1$ ，表示未来奖励的衰减程度

3.2.1 状态空间设计理论

状态空间的设计是强化学习问题建模的关键环节。一个良好设计的状态空间应当满足以下理论要求：

马尔可夫性 (Markov Property): 当前状态应包含做出最优决策所需的全部信息, 即给定当前状态, 未来的状态和奖励与过去的状态序列条件独立。数学形式化表示为:

$$P(s_{t+1}, r_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1}, r_{t+1} | s_t, a_t) \quad (3.1)$$

在网络拥塞控制场景中, 完全满足马尔可夫性是困难的, 因为网络状态可能依赖于较长的历史。Lark 通过在状态中包含 ECN 状态、CA 状态等协议栈内部状态, 在实际应用中近似满足马尔可夫性。

信息充分性 (Information Sufficiency): 状态应包含做出最优决策所需的充分信息。信息不足会导致智能体无法区分需要不同动作的情况, 从而影响学习效果。Lark 的 15 维状态空间涵盖了 TCP 协议栈提供的所有关键信息, 保证了信息充分性。

维度可扩展性 (Dimensional Scalability): 状态空间的维度不应过高, 否则会导致维度灾难 (Curse of Dimensionality) 问题。Lark 的 15 维状态空间在现有方案中属于较高维度, 但在可处理范围内。基于规则的决策机制避免了维度灾难问题。

可观测性 (Observability): 状态应能够从环境中直接或间接观测获得。Lark 的 15 个状态参数均可从 ns-3 TCP 模块的标准回调接口直接获取, 满足可观测性要求。

3.2.2 动作空间设计理论

动作空间的设计决定了智能体能够执行的控制操作范围。Lark 采用连续动作空间, 直接输出新的 ssthresh 和 cwnd 值。这种设计的理论考量如下:

控制粒度 (Control Granularity): 连续动作空间提供了最细的控制粒度, 允许智能体精确调整控制参数。相比之下, 离散动作空间 (如“增加 10%”、“减少 10%”) 限制了控制的精确性。

探索效率 (Exploration Efficiency): 连续动作空间的探索效率取决于动作的参数化形式。Lark 通过直接输出绝对值而非增量, 避免了累积误差问题。

约束满足 (Constraint Satisfaction): 动作应满足 TCP 协议的约束条件。Lark 通过在动作输出后应用边界约束, 保证输出的控制参数始终有效。

3.2.3 奖励函数设计理论

奖励函数是强化学习的核心组件, 决定了智能体的优化目标。奖励函数设计的理论要求包括:

稀疏性问题 (Sparsity Problem): 如果奖励只在特定事件 (如完成文件传输) 时给出, 智

能体可能难以学习中间步骤的价值。Lark 采用即时奖励（每次决策都给出奖励），避免了稀疏性问题。

多目标权衡 (Multi-objective Trade-off): 网络拥塞控制需要平衡多个目标（吞吐量、延迟、公平性、丢包率）。Lark 的奖励函数将吞吐量和延迟结合，通过权重参数控制优化偏好。

奖励塑形 (Reward Shaping): 适当的奖励塑形可以加速学习收敛。Lark 使用对数函数对吞吐量和延迟进行变换，实现边际效用递减，有助于平衡优化目标。

3.2.4 策略表示理论

策略 (Policy) 定义了状态到动作的映射。Lark 采用基于规则的确定性策略，而非随机策略或神经网络策略。这种选择的理论考量如下：

确定性 vs. 随机性: 确定性策略在给定状态下总是执行相同的动作，具有更好的可预测性和可解释性。随机策略可以提供更好的探索能力，但在部署阶段通常不需要探索。

参数化 vs. 非参数化: 神经网络策略是参数化策略，具有强大的表示能力，但训练和推理有计算开销。基于规则的策略是非参数化策略，计算效率高但表示能力有限。Lark 选择基于规则的策略，牺牲部分表示能力换取实时性和可解释性。

在线学习 vs. 离线设计: Lark 的规则是离线设计的，不需要在线学习过程。这避免了在线学习可能带来的不稳定性，但也限制了算法对未知环境的适应能力。

3.3 15 维观测空间详细设计

本节详细描述 Lark 算法的 15 维观测空间设计，这是 Lark 区别于现有方案的核心创新之一。

3.3.1 连接标识参数 (索引 0、3)

socketUuid (索引 0) 是套接字的唯一标识符，用于区分不同的 TCP 连接。在多流并发场景下，每个流需要维护独立的状态信息，socketUuid 是实现每流状态隔离的关键。socketUuid 在连接建立时由系统分配，在连接生命周期内保持不变。

nodeId (索引 3) 是网络节点的标识符，用于区分不同的发送端节点。在多节点仿真场景下，不同节点可能运行不同的拥塞控制算法或具有不同的网络条件，nodeId 用于识别当前决策所属的节点。

表 3.1 Lark 15 维观测空间参数定义

索引	参数名称	数据类型	说明
0	socketUuid	uint64	套接字唯一标识符
1	envType	uint64	环境类型（0= 事件驱动，1= 时间驱动）
2	simTime	uint64	仿真时间（微秒）
3	nodeId	uint64	节点标识符
4	ssthresh	uint64	慢启动阈值（字节）
5	cwnd	uint64	拥塞窗口大小（字节）
6	segSize	uint64	TCP 报文段大小（字节）
7	segmentsAcked	uint64	本次确认的报文段数量
8	bytesInFlight	uint64	在途字节数
9	lastRtt	uint64	最近往返时延（微秒）
10	minRtt	uint64	最小往返时延（微秒）
11	callbackType	uint64	回调函数类型
12	caState	uint64	拥塞算法状态
13	caEvent	uint64	拥塞算法事件
14	ecnState	uint64	ECN 状态

3.3.2 时间信息参数（索引 1、2）

envType（索引 1）是环境类型标识符，用于区分事件驱动模式（值为 0）和时间驱动模式（值为 1）。事件驱动模式在每个 TCP 回调事件发生时触发决策；时间驱动模式以固定时间间隔触发决策。Lark 默认采用事件驱动模式，以实现对网络状态变化的快速响应。

simTime（索引 2）是当前仿真时间，单位为微秒。仿真时间用于：（1）计算时间间隔，如两次回调之间的时间差；（2）估算瞬时吞吐量，如 $throughput = bytes/interval$ ；（3）跟踪 ECN 事件率，如统计最近 1 秒内的 ECN 事件次数。

3.3.3 窗口状态参数（索引 4、5）

ssthresh（索引 4）是慢启动阈值（Slow Start Threshold），单位为字节。ssthresh 是 TCP 拥塞控制的核心状态变量之一，用于区分慢启动阶段和拥塞避免阶段：当 $cwnd < ssthresh$ 时处于慢启动阶段，窗口指数增长；当 $cwnd \geq ssthresh$ 时进入拥塞避免阶段，窗口线性增长。

cwnd（索引 5）是拥塞窗口（Congestion Window）大小，单位为字节。cwnd 是 TCP 拥塞控制的另一核心状态变量，决定了发送端在未收到确认的情况下最多可以发送的数据量。cwnd 的调整是拥塞控制的核心任务。

3.3.4 传输性能参数（索引 6、7、8）

segSize（索引 6）是 TCP 报文段大小（Maximum Segment Size, MSS），单位为字节。典型值为 1460 字节（以太网 MTU 1500 字节减去 IP 报头 20 字节和 TCP 报头 20 字节）。segSize

用于将字节为单位的度量转换为报文段为单位的度量。

segmentsAcked (索引 7) 是本次确认的报文段数量。在收到累积确认 (Cumulative ACK) 或选择性确认 (Selective ACK, SACK) 时, **segmentsAcked** 表示新确认的报文段数量。**segmentsAcked** 用于计算窗口增长量, 例如在慢启动阶段, 窗口增长量为 $segmentsAcked \times MSS$ 。

bytesInFlight (索引 8) 是在途字节数 (Bytes in Flight), 即已发送但尚未确认的数据量。**bytesInFlight** 用于计算管道利用率 (Pipeline Utilization):

$$utilization = \frac{bytesInFlight}{cwnd} \quad (3.2)$$

当 **utilization** 较低时, 表明发送端未能充分利用拥塞窗口, 可能是因为应用层数据不足或接收端通告窗口限制。

3.3.5 延迟测量参数 (索引 9、10)

lastRtt (索引 9) 是最近测量的往返时延 (Round-Trip Time), 单位为微秒。RTT 是端到端延迟的直接度量, 包括传播延迟 (Propagation Delay)、排队延迟 (Queuing Delay)、处理延迟 (Processing Delay) 和传输延迟 (Transmission Delay) 四个组成部分。

minRtt (索引 10) 是历史最小往返时延, 单位为微秒。**minRtt** 被认为是网络路径的固有传播延迟 (假设其他延迟分量可以忽略), 用于估计带宽延迟积 (BDP) 和判断网络拥塞程度。RTT 膨胀比定义为:

$$inflation_ratio = \frac{lastRtt}{minRtt} \quad (3.3)$$

当 **inflation_ratio** 接近 1 时, 表明网络排队延迟较小; 当 **inflation_ratio** 较大时, 表明网络存在显著的排队延迟。

3.3.6 回调类型参数 (索引 11)

callbackType (索引 11) 标识当前回调的类型, 用于区分不同类型的事件。定义如下:

- 值为 0: GetSsThresh 回调, 表示检测到丢包事件, 需要更新慢启动阈值
- 值为 1: IncreaseWindow 回调, 表示收到正常确认, 可以增加拥塞窗口

callbackType 是多信号融合拥塞检测的关键输入之一。当 **callbackType** 为 0 时, 可以明确判断发生了丢包, 这是最高优先级的拥塞信号。

3.3.7 拥塞控制状态参数（索引 12、13、14）

caState（索引 12）是拥塞算法状态（Congestion Algorithm State），反映 TCP 拥塞控制状态机的当前状态。Linux TCP 实现定义了以下状态：

- CA_OPEN（值为 0）：开放状态，正常传输，无拥塞迹象
- CA_DISORDER（值为 1）：无序状态，收到重复确认或乱序确认
- CA_CWR（值为 2）：拥塞窗口缩减状态，响应 ECN 标记
- CA_RECOVERY（值为 3）：恢复状态，快速恢复过程中
- CA_LOSS（值为 4）：丢失状态，检测到超时丢包

caEvent（索引 13）是拥塞算法事件（Congestion Algorithm Event），表示最近发生的拥塞相关事件。Linux TCP 实现定义了以下事件：

- CA_EVENT_TX_START（值为 0）：传输开始事件
- CA_EVENT_CWND_RESTART（值为 1）：拥塞窗口重启事件
- CA_EVENT_COMPLETE_CWR（值为 2）：CWR 完成事件
- CA_EVENT_LOSS（值为 3）：丢失事件
- CA_EVENT_ECN_NO_CE（值为 4）：无 CE 标记的 ECN 事件
- CA_EVENT_ECN_IS_CE（值为 5）：有 CE 标记的 ECN 事件

ecnState（索引 14）是显式拥塞通知状态（ECN State），反映 ECN 协商和标记的状态。定义如下：

- ECN_DISABLED（值为 0）：ECN 禁用
- ECN_IDLE（值为 1）：ECN 空闲
- ECN_CE_RCVD（值为 2）：收到 CE 标记
- ECN_SENDING_ECE（值为 3）：正在发送 ECE 标志
- ECN_ECE_RCVD（值为 4）：收到 ECE 标志
- ECN_CWR_SENT（值为 5）：已发送 CWR 标志

这三个拥塞控制状态参数是 Lark 观测空间的核心创新。现有强化学习拥塞控制方案通常不包含这些协议栈内部状态，导致智能体对网络拥塞状况的感知不完整。Lark 通过将 `caState`、`caEvent`、`ecnState` 纳入观测空间，使智能体能够感知 TCP 协议栈的完整状态信息，从而做出更精准的控制决策。

3.4 多信号融合拥塞检测算法

Lark 算法的核心创新之一是多信号融合拥塞检测方法。该方法综合分析多种拥塞信号，采用分层优先级机制和频率过滤策略，准确判断网络拥塞状态并量化拥塞严重程度。

3.4.1 拥塞信号分类

Lark 将拥塞信号分为四个类别：

(1) **显式丢包信号**：当 `callbackType` 为 0 (`GetSsThresh` 回调) 时，表明检测到了丢包事件。丢包是最明确的拥塞信号，通常意味着网络缓冲区已经溢出。

(2) **ECN 拥塞经历信号**：当 `ecnState` 为 `ECN_CE_RCVD` (值为 2) 或 `caEvent` 为 `CA_EVENT_ECN_IS_C` (值为 5) 时，表明数据包被网络设备标记为拥塞经历。ECN 信号是早期拥塞信号，表明网络设备的队列已经超过阈值，但尚未溢出。

(3) **拥塞控制状态机信号**：当 `caState` 为 `CA_CWR`、`CA_RECOVERY` 或 `CA_LOSS` 时，表明 TCP 协议栈的拥塞控制状态机已经检测到拥塞迹象并进入相应状态。

(4) **延迟膨胀信号**：当 RTT 膨胀比 ($\text{lastRtt}/\text{minRtt}$) 超过阈值时，表明网络存在显著的排队延迟，可能处于拥塞状态。

3.4.2 分层优先级机制

不同类型的拥塞信号具有不同的可靠性和紧迫程度，Lark 采用分层优先级机制进行处理：

第一优先级：显式丢包检测

当 `callbackType` 为 0 时，明确判定发生丢包。Lark 进一步区分快速重传检测到的丢包 (`caState` $\neq$ `CA_LOSS`) 和超时导致的丢包 (`caState` = `CA_LOSS`)，分别采用不同的响应策略。这是最高优先级的拥塞信号，一旦检测到丢包，无需进一步检查其他信号。

第二优先级：ECN 拥塞经历信号检测

当 `ecnState` 为 `ECN_CE_RCVD` (收到 CE 标记) 或 `ECN_ECE_RCVD` (收到 ECE 回显) 时，判定发生 ECN 拥塞。ECN 拥塞采用较温和的响应策略 (保留因子 0.85)，因为 ECN 是早期拥塞信号，网络尚未发生实际丢包。

Algorithm 1 显式丢包检测**Require:** 回调类型 *callbackType*, 拥塞算法状态 *caState*

```

1: if callbackType = 0 then                                     ▷ GetSsThresh 回调
2:   is_congested  $\leftarrow$  True
3:   if caState = CA_LOSS then
4:     congestion_type  $\leftarrow$  TIMEOUT
5:   else
6:     congestion_type  $\leftarrow$  LOSS
7:   end if
8:   return (is_congested, congestion_type)
9: end if

```

Algorithm 2 ECN 拥塞经历信号检测**Require:** ECN 状态 *ecnState*

```

1: if ecnState  $\in$  {ECN_CE_RCVD, ECN_ECE_RCVD} then
2:   is_congested  $\leftarrow$  True
3:   congestion_type  $\leftarrow$  ECN
4:   return (is_congested, congestion_type)
5: end if

```

3.4.3 频率过滤策略

为了避免对瞬态扰动的过度响应, Lark 采用频率过滤策略, 将以下信号排除在拥塞触发条件之外:

(1) **单次 ECN 事件:** 单次 ECN 标记可能是由于队列瞬时波动导致, 不能作为持续性拥塞的判断依据。Lark 通过跟踪 ECN 事件率, 只有当事件率超过阈值时才触发拥塞响应。

(2) **ECN 回显状态:** *ecnState* 为 *ECN_ECE_RCVD* 表示收到了接收端的 ECN 回显, 但此时发送端可能已经响应了该拥塞通知。为避免重复响应, 不将其作为独立的拥塞触发条件。

(3) **拥塞窗口缩减状态:** *caState* 为 *CA_CWR* 表示发送端正在响应 ECN 标记并缩减窗口。此时拥塞响应已经在进行中, 不需要再次触发。

(4) **恢复状态:** *caState* 为 *CA_RECOVERY* 表示正处于快速恢复过程中。快速恢复是对丢包的响应机制, 不应重复触发拥塞响应。

(5) **RTT 膨胀:** 虽然 RTT 膨胀是拥塞的间接指示, 但 RTT 测量易受多种因素干扰, 单独依靠 RTT 膨胀判断拥塞可能导致误判。Lark 将 RTT 膨胀用于自适应参数调优, 而非直接用于拥塞触发。

3.4.4 完整拥塞检测算法

综合上述机制, Lark 的完整多信号融合拥塞检测算法如下:

Algorithm 3 多信号融合拥塞检测算法**Require:** 观测状态 *obs*, 流状态 *state***Ensure:** 是否拥塞 *is_congested*, 拥塞类型 *type*

```

1:                                     ▷ 第一优先级：显式丢包检测
2: if obs.callbackType = 0 then                                     ▷ GetSsThresh 回调
3:   state.loss_count  $\leftarrow$  state.loss_count + 1
4:   if obs.caState = CA_LOSS then
5:     return (True, TIMEOUT)                                     ▷ 超时丢包
6:   else
7:     return (True, LOSS)                                       ▷ 快速重传丢包
8:   end if
9: end if
10:                                     ▷ 第二优先级：ECN 拥塞经历检测
11: if obs.ecnState  $\in$  {ECN_CE_RCVD, ECN_ECE_RCVD} then
12:   state.ecn_count  $\leftarrow$  state.ecn_count + 1
13:   return (True, ECN)
14: end if
15:                                     ▷ 未检测到拥塞
16: return (False, NONE)

```

3.5 差异化拥塞响应机制

Lark 算法针对不同类型的拥塞信号采用差异化的响应策略，核心是差异化窗口保留因子（Window Retention Factor）机制。

3.5.1 窗口保留因子设计

窗口保留因子 ρ 定义为拥塞响应后新窗口与原窗口的比值：

$$new_wnd = \rho \cdot cwnd \quad (3.4)$$

传统算法（如 TCP Reno、CUBIC）对所有拥塞信号采用统一的窗口保留因子（通常为 0.5，即窗口减半）。这种“一刀切”的策略无法区分不同类型和程度的拥塞，在轻度拥塞时响应过度，在严重拥塞时响应不足。

Lark 根据拥塞类型设计了差异化的窗口保留因子：

表 3.2 差异化窗口保留因子

拥塞类型	保留因子	设计理由
丢包（Loss）	0.70	丢包是严重拥塞信号，需要较大幅度降低窗口
ECN 标记（ECN）	0.85	ECN 是早期信号，保留较大窗口维持吞吐量
超时丢失（Timeout）	0.50	超时表示最严重拥塞，需最大幅度窗口缩减

3.5.2 差异化响应的理论依据

差异化窗口保留因子的设计基于以下理论分析：

(1) 拥塞信号的时序特性

ECN 标记是最早的拥塞信号，通常在队列长度超过低阈值时触发，此时网络可能只是轻度拥塞。丢包是较晚的拥塞信号，通常在队列溢出时发生，表明网络已经严重拥塞。超时丢包是最晚也最严重的拥塞信号，通常意味着多个数据包连续丢失或网络路径严重中断，需要最大力度的窗口缩减。

根据时序关系和严重程度，Lark 设计了三级差异化响应：ECN 标记采用最温和的响应（保留因子 0.85），丢包采用较激进的响应（保留因子 0.70），超时采用最激进的响应（保留因子 0.50）。这种阶梯式的响应力度设计确保了对不同严重程度拥塞的精确匹配。

(2) 信号可靠性分析

丢包是最可靠的拥塞信号，几乎不会出现误报（False Positive）。ECN 标记的可靠性取决于网络设备的配置，如果阈值设置不当，可能出现误报。基于可靠性考虑，对丢包信号的响应应更加坚决（较低的保留因子），对 ECN 信号的响应可以更加保守（较高的保留因子）。

(3) 吞吐量-延迟权衡

较低的窗口保留因子能够更快地缓解拥塞、降低延迟，但会导致吞吐量下降。较高的窗口保留因子能够维持较高的吞吐量，但可能延长拥塞持续时间。Lark 的差异化设计试图在吞吐量和延迟之间取得最优平衡。

3.5.3 慢启动阈值更新

除了拥塞窗口，拥塞响应还需要更新慢启动阈值。Lark 采用以下策略：

$$new_ssthresh = \max(new_cwnd, BDP) \quad (3.5)$$

这个设计的考虑是：（1）新的 $ssthresh$ 应不小于新的 $cwnd$ ，否则会立即进入慢启动阶段，导致窗口增长过快；（2） $ssthresh$ 设为 BDP 的估计值，允许在拥塞缓解后快速恢复到网络的带宽延迟积水平。

此外，Lark 引入了连续缩减保护机制（Consecutive Decrease Protection）。在实际网络中，连续的拥塞事件可能导致窗口持续缩减直至极小值，严重影响吞吐量。Lark 设置最大连续缩减次数为 3 次：当连续拥塞响应超过 3 次时，保持当前窗口不再缩减，等待窗口正常增长后重置计数器。这一机制有效防止了窗口的过度收缩，保障了最低传输性能。

3.5.4 差异化拥塞响应算法

Algorithm 4 差异化拥塞响应算法

Require: 当前窗口 $cwnd$, 当前阈值 $ssthresh$, 拥塞类型 $type$, 连续缩减计数 $consec_dec$

Ensure: 新窗口 new_cwnd , 新阈值 $new_ssthresh$

```

1:  $consec\_dec \leftarrow consec\_dec + 1$ 
2:                                     ▷ 连续缩减保护：超过 3 次后保持窗口
3: if  $consec\_dec > 3$  then
4:   return ( $cwnd, cwnd$ )                                     ▷ 保持当前窗口不再缩减
5: end if
6:                                     ▷ 根据拥塞类型选择窗口保留因子
7: if  $type = TIMEOUT$  then
8:    $\rho \leftarrow 0.50$                                      ▷ 超时：最严厉响应
9: else if  $type = LOSS$  then
10:   $\rho \leftarrow 0.70$                                      ▷ 丢包：较激进响应
11: else if  $type = ECN$  then
12:   $\rho \leftarrow 0.85$                                      ▷ ECN：温和响应
13: else
14:   $\rho \leftarrow 1.0$                                      ▷ 无拥塞，不调整
15: end if
16:                                     ▷ 计算新窗口
17:  $new\_cwnd \leftarrow \lfloor \rho \times cwnd \rfloor$ 
18:                                     ▷ 应用窗口下限约束
19:  $new\_cwnd \leftarrow \max(new\_cwnd, 4 \times MSS)$ 
20:                                     ▷ 计算新慢启动阈值：取新窗口与 BDP 的较大值
21:  $new\_ssthresh \leftarrow \max(new\_cwnd, BDP)$ 
22: return ( $new\_cwnd, new\_ssthresh$ )

```

3.6 强化学习驱动的自适应参数调优

Lark 算法实现了强化学习驱动的自适应参数调优机制，根据多种因素动态调整乘性增加因子（Multiplicative Increase Factor, α ）。

3.6.1 乘性增加因子的作用

在 Lark 的融合控制策略中，乘性增加因子 α 控制拥塞避免阶段的窗口增长速度：

$$target_cwnd = \max(\alpha \times BDP, cwnd) + \gamma \times MSS \quad (3.6)$$

α 的取值决定了窗口增长的激进程度：较大的 α 导致更激进的窗口增长，能够更快地利用可用带宽，但也更容易导致拥塞；较小的 α 导致更保守的窗口增长，网络更稳定，但带宽利用效率较低。

3.6.2 多因素调整机制

Lark 根据以下因素动态调整 α :

(1) 基于 RTT 膨胀比的三级梯度调整

RTT 膨胀比反映网络排队延迟程度:

$$inflation_ratio = \frac{lastRtt}{minRtt} \quad (3.7)$$

调整规则采用三级梯度响应:

- 当 $inflation_ratio < 1.3$ 时, 排队延迟极小, 大幅增大 α : $\alpha \leftarrow \alpha + 0.03$
- 当 $1.3 \leq inflation_ratio < 2.0$ 时, 排队延迟适中, 温和增大 α : $\alpha \leftarrow \alpha + 0.01$
- 当 $inflation_ratio > 3.0$ 时, 排队延迟过大, 减小 α : $\alpha \leftarrow \alpha - 0.02$

相比简单的二级阈值判断, 三级梯度响应机制能够更精细地适配不同程度的网络排队状态, 在低排队环境中快速提升带宽利用率, 在高排队环境中及时降低发送激进度。

(2) 基于奖励信号的趋势调整

Lark 利用环境反馈的奖励信号 (Reward) 进行策略效果评估。采用指数移动平均 (Exponential Moving Average, EMA) 对奖励信号进行平滑处理:

$$R_{ema} \leftarrow (1 - \eta) \times R_{ema} + \eta \times reward \quad (3.8)$$

其中 $\eta = 0.1$ 为 EMA 平滑因子。根据奖励趋势调整 α :

- 当 $R_{ema} > 2.0$ 时, 表明当前策略效果良好, 增大 α : $\alpha \leftarrow \alpha + 0.01$
- 当 $R_{ema} < -5.0$ 时, 表明当前策略效果不佳, 减小 α : $\alpha \leftarrow \alpha - 0.01$

奖励信号的引入使得 Lark 能够从全局控制效果的角度评估参数调整的方向, 弥补了仅依赖 RTT 膨胀比等局部指标的不足。

(3) 基于连续增长趋势的调整

当窗口连续增长超过 5 次时, 表明网络状况持续良好, 给予稳定增长奖励:

$$\alpha \leftarrow \alpha + 0.02 \quad \text{如果连续增长次数} > 5 \quad (3.9)$$

3.6.3 参数边界约束

为保证控制策略的稳定性， α 的取值被限制在预设范围内：

$$\alpha \in [\alpha_{min}, \alpha_{max}] = [1.05, 1.50] \quad (3.10)$$

α 的默认初始值为 1.20。下界 1.05 保证了即使在最保守的情况下，目标窗口仍略大于 BDP，避免带宽利用不足；上界 1.50 限制了最激进的增长率，防止窗口过快膨胀导致拥塞。这个设计允许 α 在较大范围内动态调整，以适应不同的网络环境。

3.6.4 自适应参数调优算法

Algorithm 5 自适应参数调优算法

Require: 当前 α ，观测状态 obs ，奖励信号 $reward$ ，历史状态 $history$

Ensure: 更新后的 α

```

1:                                     ▷ 因素 1: 基于 RTT 膨胀比的三级梯度调整
2:  $inflation\_ratio \leftarrow obs.lastRtt / obs.minRtt$ 
3: if  $inflation\_ratio < 1.3$  then
4:    $\alpha \leftarrow \alpha + 0.03$                                      ▷ 排队极小，大幅增大
5:    $history.consecutive\_growth \leftarrow history.consecutive\_growth + 1$ 
6: else if  $inflation\_ratio < 2.0$  then
7:    $\alpha \leftarrow \alpha + 0.01$                                      ▷ 排队适中，温和增大
8: else if  $inflation\_ratio > 3.0$  then
9:    $\alpha \leftarrow \alpha - 0.02$                                      ▷ 排队过大，减小
10:   $history.consecutive\_growth \leftarrow 0$ 
11: end if
12:                                     ▷ 因素 2: 基于奖励信号 EMA 的趋势调整
13:  $R_{ema} \leftarrow 0.9 \times R_{ema} + 0.1 \times reward$ 
14: if  $R_{ema} > 2.0$  then
15:    $\alpha \leftarrow \alpha + 0.01$                                      ▷ 策略效果良好
16: else if  $R_{ema} < -5.0$  then
17:    $\alpha \leftarrow \alpha - 0.01$                                      ▷ 策略效果不佳
18: end if
19:                                     ▷ 因素 3: 基于连续稳定增长的奖励
20: if  $history.consecutive\_growth > 5$  then
21:    $\alpha \leftarrow \alpha + 0.02$ 
22: end if
23:                                     ▷ 应用边界约束
24:  $\alpha \leftarrow \max(\min(\alpha, 1.50), 1.05)$ 
25: return  $\alpha$ 

```

3.7 融合控制策略

Lark 采用融合控制策略计算目标拥塞窗口，将基于带宽延迟积（BDP）估计的速率控制与基于当前窗口的保守控制相结合。

3.7.1 带宽延迟积估计

带宽延迟积（Bandwidth-Delay Product, BDP）是网络链路能够容纳的最大在途数据量，是拥塞控制的重要参考指标：

$$BDP = Bandwidth \times RTT \quad (3.11)$$

Lark 通过以下方式估计 BDP：

$$BDP = \max_{i \in W} (delivery_rate_i) \times minRtt \quad (3.12)$$

其中， $\max_{i \in W} (delivery_rate_i)$ 是窗口化最大值过滤（Windowed Max-filter）方法估计的瓶颈带宽，窗口大小 $W = 20$ 个样本点。单个样本的投递率（Delivery Rate）计算为：

$$delivery_rate = \frac{segmentsAcked \times segSize}{lastRtt} \quad (3.13)$$

相比简单地使用历史最大吞吐量，窗口化最大值过滤方法的优势在于：（1）通过限制采样窗口大小，能够适应网络带宽的动态变化；（2）取窗口内最大值能够过滤 RTT 波动导致的低估；（3）计算复杂度为 $O(1)$ （每次插入新样本时更新最大值）。

吞吐量的跟踪采用指数移动平均方式：

$$throughput_ema = 0.9 \times throughput_ema + 0.1 \times delivery_rate \quad (3.14)$$

3.7.2 慢启动阶段策略

在慢启动阶段（ $cwnd < ssthresh$ ），Lark 采用指数增长策略以快速探测可用带宽：

$$target_cwnd = 2 \times BDP \quad (3.15)$$

$$increment = segmentsAcked \times MSS \quad (3.16)$$

当窗口远低于 BDP 时, 采用加速增长:

$$\text{如果 } cwnd < 0.3 \times BDP, \quad increment = 2 \times segmentsAcked \times MSS \quad (3.17)$$

慢启动目标设为 $2 \times BDP$ 而非更大值 (如 $3 \times BDP$), 是为了避免慢启动阶段过度膨胀导致突发丢包。当窗口达到目标值后, 自动退出慢启动进入拥塞避免阶段。

3.7.3 拥塞避免阶段策略

在拥塞避免阶段 ($cwnd \geq ssthresh$), Lark 采用稳健的窗口增长策略:

$$target_cwnd = \max(\alpha \times BDP, cwnd) + \gamma \times MSS \quad (3.18)$$

其中, α 是乘性增加因子 (通过自适应调优动态调整, 基准值 1.20), $\gamma = 2.0$ 是加性增加因子, 表示每个 ACK 事件增加 2 个 MSS 的窗口增量。较大的 γ 值使得 Lark 在非拥塞阶段能够更快速地增加窗口, 充分利用可用带宽。

融合控制策略还考虑管道利用率因素。管道利用率定义为:

$$utilization = \frac{bytesInFlight}{cwnd} \quad (3.19)$$

调整规则:

- 当 $utilization < 0.7$ 时, 额外增加 $1 \times MSS$
- 当 $utilization < 0.4$ 时, 额外增加 $2 \times MSS$ (累计 $3 \times MSS$)

这个设计确保了在管道未充分利用时能够快速增加窗口, 应对应用层突发数据到达或接收窗口临时限制解除等场景。

3.7.4 窗口边界约束

为保证窗口大小的合理性, Lark 对计算结果应用边界约束:

$$cwnd \in [4 \times MSS, \max(10 \times BDP, 200 \times MSS)] \quad (3.20)$$

下界 $4 \times MSS$ 是 TCP 的最小窗口大小, 保证基本的传输能力。上界设为 BDP 的 10 倍或 200 个 MSS 的较大值, 提供充裕的窗口增长空间以充分利用管道容量, 同时避免窗口无限增长导致网络不稳定。

3.7.5 融合控制策略算法

Algorithm 6 融合控制策略算法

Require: 观测状态 obs , 自适应参数 α , $\gamma = 2.0$, BDP 估计 bdp

Ensure: 新窗口 new_cwnd

```

1:                                     ▷ 重置连续缩减计数器
2:  $consecutive\_decreases \leftarrow 0$ 
3:                                     ▷ 计算管道利用率
4:  $utilization \leftarrow obs.bytesInFlight / obs.cwnd$ 
5:                                     ▷ 判断当前阶段
6: if  $obs.cwnd < obs.ssthresh$  and  $in\_slow\_start$  then
7:                                     ▷ 慢启动阶段
8:    $target \leftarrow 2 \times bdp$ 
9:   if  $obs.cwnd < 0.3 \times bdp$  then
10:     $increment \leftarrow 2 \times obs.segmentsAacked \times obs.segSize$ 
11:   else
12:     $increment \leftarrow obs.segmentsAacked \times obs.segSize$ 
13:   end if
14:    $new\_cwnd \leftarrow \min(obs.cwnd + increment, target)$ 
15: else
16:                                     ▷ 拥塞避免阶段
17:    $target\_rate \leftarrow \alpha \times bdp$ 
18:    $\gamma\_bytes \leftarrow \gamma \times obs.segSize$ 
19:    $new\_cwnd \leftarrow \lfloor \max(target\_rate, obs.cwnd) + \gamma\_bytes \rfloor$ 
20:                                     ▷ 管道利用率感知加速
21:   if  $utilization < 0.4$  then
22:     $new\_cwnd \leftarrow new\_cwnd + 3 \times obs.segSize$ 
23:   else if  $utilization < 0.7$  then
24:     $new\_cwnd \leftarrow new\_cwnd + 1 \times obs.segSize$ 
25:   end if
26: end if
27:                                     ▷ 应用边界约束
28:  $min\_cwnd \leftarrow 4 \times obs.segSize$ 
29:  $max\_cwnd \leftarrow \max(10 \times bdp, 200 \times obs.segSize)$ 
30:  $new\_cwnd \leftarrow \max(\min(new\_cwnd, max\_cwnd), min\_cwnd)$ 
31: return  $new\_cwnd$ 

```

3.8 每流独立状态管理

Lark 设计了每流独立的状态管理机制, 为每个 TCP 连接维护独立的状态数据结构, 支持多流并发场景下的差异化控制。

3.8.1 状态数据结构

每个 TCP 连接维护以下状态数据, 采用层次化组织方式:

(1) **历史度量缓冲区**: 带宽估计采用窗口化最大值过滤, 维护一个容量为 20 个样本点的投递率 (Delivery Rate) 滑动窗口队列, 取窗口内最大值作为瓶颈带宽估计。RTT 跟踪维护一个容量为 40 个样本点的最近 RTT 队列, 同时维护全局最小 RTT 作为传播延迟估计。BDP 由最大带宽与最小 RTT 的乘积得到。

(2) **ECN 事件跟踪数据**: ECN 事件跟踪模块维护 ECN 事件计数器, 记录丢包事件计数和 ECN 标记事件计数, 用于统计分析拥塞事件的分布特征。

(3) **性能指标数据**: 性能指标模块维护吞吐量的指数移动平均值 (EMA, 平滑因子 0.1), 用于跟踪流级别的性能变化趋势。

(4) **自适应参数数据**: 自适应参数模块维护乘性增加因子 α (取值范围 [1.05, 1.50], 默认值 1.20)、加性增加因子 γ (默认值 2.0)、连续缩减计数器 (用于连续缩减保护)、连续增长计数器 (用于稳定增长奖励) 和慢启动阶段标记。

3.8.2 状态查找与更新

Lark 使用 socketUuid 作为键, 通过哈希表实现 $O(1)$ 时间复杂度的状态查找和更新。哈希表的选择基于查找效率、动态扩展和内存效率三方面考量。

Algorithm 7 流状态查找与更新

```

1: function LOOKUPFLOWSTATE(manager, socket_uuid)
2:   hash  $\leftarrow$  ComputeHash(socket_uuid)
3:   bucket  $\leftarrow$  manager.buckets[hash mod manager.num_buckets]
4:   for each entry in bucket do
5:     if entry.key = socket_uuid then
6:       return entry.value
7:     end if
8:   end for
9:   return null                                     ▷ 未找到
10: end function
11: function UPDATEFLOWSTATE(manager, socket_uuid, obs)
12:   state  $\leftarrow$  GetOrCreateFlowState(manager, socket_uuid)
13:   state.last_update_time  $\leftarrow$  obs[2]
14:   UpdateMetrics(state.metrics_buffer, obs)
15:   if IsECNEvent(obs) then
16:     state.ecn_tracker.RecordEvent(obs[2])
17:   end if
18:   UpdatePerformanceMetrics(state.performance, obs)
19: end function

```

3.8.3 状态生命周期管理

TCP 连接的生命周期包括建立、传输和关闭三个阶段。Lark 的状态管理遵循延迟初始化、增量更新、垃圾回收和状态持久化四个原则。

Algorithm 8 流状态垃圾回收

```

1: function GARBAGECOLLECT(manager, current_time, inactive_threshold)
2:   to_remove  $\leftarrow$  EmptyList()
3:   for each (uuid, state) in manager.flow_states do
4:     idle_time  $\leftarrow$  current_time - state.last_update_time
5:     if idle_time > inactive_threshold then
6:       to_remove.Append(uuid)
7:     end if
8:   end for
9:   for each uuid in to_remove do
10:    manager.flow_states.Remove(uuid)
11:    FreeResources(uuid)
12:   end for
13:   return |to_remove|
14: end function

```

3.9 实现细节与优化技术

本节详细介绍 Lark 算法的具体实现细节和优化技术。

3.9.1 系统架构设计

Lark 算法的系统架构遵循模块化设计原则，采用分层结构实现各功能模块的解耦。整体架构分为仿真层、环境层、决策层和通信层四个层次。仿真层负责网络拓扑构建和数据包传输仿真；环境层作为强化学习与网络仿真的桥梁；决策层实现 Lark 算法的核心逻辑，遵循状态隔离、分层决策和实时性原则；通信层负责仿真进程和决策进程之间的数据交换。

3.9.2 核心算法伪代码

Lark 智能体的主控制循环负责协调各功能模块，处理来自环境的观测并输出控制动作。

3.9.3 数据结构设计

Lark 算法采用高效的数据结构支持其核心功能。循环队列（Circular Buffer）用于存储历史度量值，具有固定内存占用、 $O(1)$ 时间复杂度的插入和删除操作、自动保持时间局部性

Algorithm 9 Lark 系统初始化流程

Require: 网络拓扑配置 $topology$, 仿真参数 $params$ **Ensure:** 初始化完成的 Lark 系统

```

1:  $simulator \leftarrow \text{CreateSimulator}(params.duration)$ 
2:  $network \leftarrow \text{BuildTopology}(topology)$ 
3:  $\text{ConfigureQueues}(network, params.queue\_size)$ 
4:  $env \leftarrow \text{CreateOpenGymEnvironment}()$ 
5:  $\text{RegisterCallbacks}(env, network.tcp\_sockets)$ 
6:  $\text{ConfigureObservationSpace}(env, 15)$ 
7:  $\text{ConfigureActionSpace}(env, 2)$  ▷ ssthresh 和 cwnd
8:  $agent \leftarrow \text{CreateLarkAgent}()$ 
9:  $agent.flow\_states \leftarrow \text{EmptyHashMap}()$ 
10:  $agent.\alpha_{default} \leftarrow 1.20$ 
11:  $channel \leftarrow \text{CreateMessageChannel}()$ 
12:  $\text{ConnectEnvAndAgent}(env, agent, channel)$ 
13:  $simulator.Run()$ 

```

Algorithm 10 Lark 主控制循环

Require: 环境接口 env , 最大迭代次数 $max_iterations$ **Ensure:** 控制结果序列

```

1:  $iteration \leftarrow 0$ 
2:  $flow\_manager \leftarrow \text{InitFlowStateManager}()$ 
3: while  $iteration < max\_iterations$  and not  $env.IsDone()$  do
4:    $obs \leftarrow env.GetObservation()$ 
5:    $socket\_uuid \leftarrow obs[0]$ 
6:    $flow\_state \leftarrow flow\_manager.GetOrCreate(socket\_uuid)$ 
7:    $\text{UpdateFlowState}(flow\_state, obs)$ 
8:    $(is\_congested, cong\_type, severity) \leftarrow \text{DetectCongestion}(obs, flow\_state)$ 
9:   if  $is\_congested$  then
10:     $action \leftarrow \text{CongestionResponse}(obs, cong\_type, flow\_state)$ 
11:   else
12:     $\text{UpdateAdaptiveParams}(flow\_state, obs)$ 
13:     $action \leftarrow \text{FusionControl}(obs, flow\_state)$ 
14:   end if
15:    $env.ExecuteAction(action)$ 
16:    $iteration \leftarrow iteration + 1$ 
17: end while

```

等优势。**ECN 事件跟踪器**用于计算 ECN 事件率，维护时间戳序列支持快速查询。**性能指标聚合器**负责维护流级别的性能统计信息。

3.9.4 性能优化技术

为了提高 Lark 算法的执行效率，满足实时决策的要求，本文采用了多种性能优化技术：

(1) **延迟初始化策略**：流状态采用延迟初始化策略，只在首次访问时创建状态数据结构。

Algorithm 11 延迟初始化状态获取

```

1: function GETORCREATEFLOWSTATE(manager, socket_uuid)
2:   if socket_uuid  $\notin$  manager.flow_states then
3:     new_state  $\leftarrow$  CreateEmptyFlowState()
4:     new_state. $\alpha$   $\leftarrow$  1.20
5:     new_state.min_rtt  $\leftarrow$   $\infty$ 
6:     new_state.max_throughput  $\leftarrow$  0
7:     new_state.consecutive_decreases  $\leftarrow$  0
8:     new_state.consecutive_increases  $\leftarrow$  0
9:     new_state.in_slow_start  $\leftarrow$  True
10:    manager.flow_states[socket_uuid]  $\leftarrow$  new_state
11:   end if
12:   return manager.flow_states[socket_uuid]
13: end function

```

(2) **增量计算优化**：吞吐量、BDP 等指标的计算采用增量方式，时间复杂度为 $O(1)$ 。

(3) **查找表优化**：窗口保留因子使用查找表实现，避免条件分支带来的 CPU 流水线停顿。

(4) **批量状态更新**：将多次更新操作合并为一次，减少函数调用开销和内存访问次数。

(5) **缓存友好设计**：将频繁访问的字段放置在相邻的内存位置，提高 CPU 缓存的命中率。

3.9.5 多线程并行化架构

为了提高仿真效率，Lark 系统采用了多线程并行化仿真架构。拓扑分区策略将网络拓扑划分为多个逻辑分区，遵循负载均衡、最小化跨分区通信和保持拓扑局部性原则。并行事件调度采用保守同步协议，通过空消息机制确保因果一致性。

实验表明，在 8 核 CPU 上，多线程并行化可以获得 3–5 倍的仿真加速，显著提高了大规模网络仿真的效率。

3.9.6 正确性保证机制

Lark 算法的实现包含参数边界检查、状态一致性检查和异常处理机制三种正确性保证机制。所有控制参数都经过边界检查，确保输出值在有效范围内；在每次决策后检查 TCP 状态

Algorithm 12 保守同步并行仿真

Require: 分区集合 $partitions$, 终止时间 end_time

```

1: for each partition  $p$  in  $partitions$  do
2:    $p.lvt \leftarrow 0$ 
3:    $p.lookahead \leftarrow \text{ComputeLookahead}(p)$ 
4: end for
5: while  $\exists p : p.lvt < end\_time$  do
6:   并行执行  $\forall p \in partitions$ :
7:      $safe\_time \leftarrow \text{ComputeSafeTime}(p)$ 
8:     while  $p.event\_queue.PeekTime() \leq safe\_time$  do
9:        $event \leftarrow p.event\_queue.Dequeue()$ 
10:       $\text{ProcessEvent}(p, event)$ 
11:     end while
12:      $\text{SendNullMessages}(p, p.lvt + p.lookahead)$ 
13:   结束并行执行
14:    $\text{Barrier}()$ 
15: end while

```

的一致性；在检测到异常情况时能够安全地回退到保守策略。

3.10 本章小结

本章详细介绍了 Lark 算法的设计与实现，包括：

（1）系统架构：由网络仿真模块、拥塞控制环境模块、智能决策模块和进程间通信模块组成，实现了基于 ns-3 和 OpenGym 的端到端强化学习拥塞控制框架。

（2）15 维观测空间：涵盖连接标识、时间信息、窗口状态、传输指标、延迟测量、回调类型和拥塞控制状态等多个维度，为强化学习智能体提供完整的环境感知能力。

（3）多信号融合拥塞检测：采用分层优先级机制和频率过滤策略，综合分析丢包、ECN、拥塞状态等多种信号，准确判断拥塞状态并量化严重程度。

（4）差异化拥塞响应：针对不同拥塞类型采用不同的窗口保留因子（丢包 0.70、ECN 0.85、超时 0.50），并引入连续缩减保护机制（最多 3 次连续缩减后保持窗口），实现对拥塞信号的精准响应。

（5）自适应参数调优：根据 RTT 膨胀比的三级梯度响应、奖励信号 EMA 趋势和连续稳定增长动态调整乘性增加因子 $\alpha \in [1.05, 1.50]$ （基准值 1.20），实现控制策略对网络环境的实时适配。

（6）融合控制策略：按公式 $V_t = \max(\alpha \times BDP, W) + \gamma \times MSS$ ($\gamma = 2.0$) 将基于 BDP 窗口化最大值过滤估计的速率控制与基于当前窗口的保守控制相结合，在慢启动阶段以 $2 \times BDP$ 为目标，在拥塞避免阶段引入管道利用率感知的加速增长机制。

（7）每流独立状态管理：为每个 TCP 连接维护独立的状态数据，支持多流并发场景下的

差异化控制。

这些设计共同构成了 Lark 算法的完整技术方案，为实现”高吞吐、低延迟”的设计目标奠定了基础。

第四章 实验评估

本章介绍 Lark 算法的实验评估，包括实验环境配置、评测指标、对比算法、实验场景设计以及详细的实验结果分析。

4.1 实验环境配置

4.1.1 硬件环境

实验在以下硬件环境中进行：

- CPU: Intel Core i7-10700K @ 3.80GHz, 8 核 16 线程
- 内存: 32GB DDR4 3200MHz
- 存储: 512GB NVMe SSD
- 操作系统: Ubuntu 20.04 LTS

4.1.2 软件环境

实验使用以下软件工具：

- 网络仿真器: ns-3.40
- 强化学习框架: OpenGym (ns3-gym)
- Python 版本: Python 3.8.10
- 依赖库: NumPy 1.21.0, ZeroMQ 4.3.4, Protocol Buffers 3.17.3

4.1.3 网络拓扑

实验采用经典的哑铃型 (Dumbbell) 网络拓扑，如图4.1所示。该拓扑包括：

- 左侧叶子节点（发送端）： n 个节点，默认 $n = 3$
- 右侧叶子节点（接收端）： n 个节点
- 两个核心路由器：R0（左侧）和 R1（右侧）

- 接入链路：连接叶子节点和路由器
- 瓶颈链路：连接两个路由器

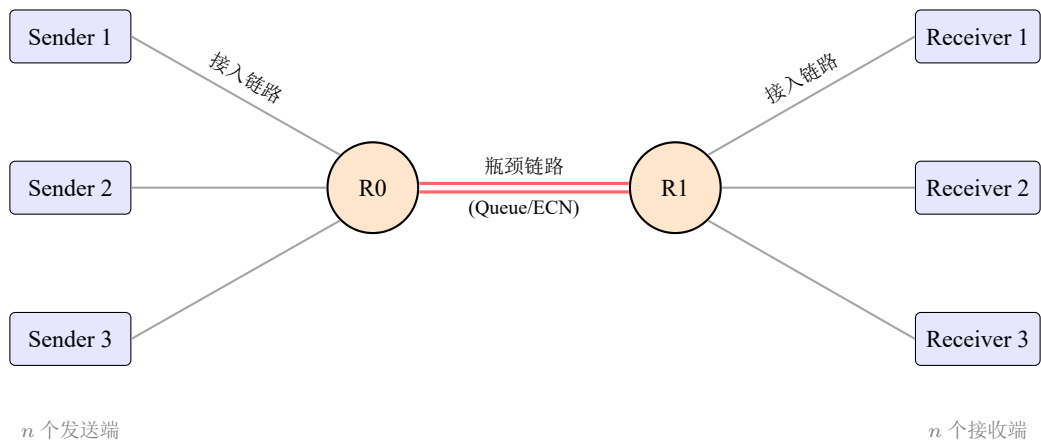


图 4.1 哑铃型（Dumbbell）实验网络拓扑

瓶颈链路采用 DropTail 队列管理策略，队列长度设置为 BDP 的 2 倍。启用 ECN，确保 Lark 协议能够充分利用 ECN 信号进行精细化拥塞控制。

4.1.4 仿真参数

默认仿真参数配置如表4.1所示：

表 4.1 默认仿真参数

参数	默认值	说明
仿真时长	20 秒	每次实验的仿真时间
叶子节点数	3	发送端和接收端各 3 个节点
随机种子	42	保证实验可重复性
TCP 段大小	1460 字节	以太网 MTU 下的典型 MSS
初始窗口	10 个 MSS	根据 RFC 6928

4.2 评测指标

本文采用以下指标评估拥塞控制算法的性能：

4.2.1 吞吐量（Throughput）

吞吐量定义为单位时间内成功传输的数据量，是衡量网络带宽利用效率的核心指标：

$$Throughput = \frac{TotalBytesReceived}{SimulationTime}$$

(单位：Mbps)

(4.1)

对于多流场景，我们计算所有流的聚合吞吐量和每流平均吞吐量：

$$AggregateThroughput = \sum_{i=1}^n Throughput_i \quad (4.2)$$

$$AverageThroughput = \frac{AggregateThroughput}{n} \quad (4.3)$$

4.2.2 延迟 (Latency)

延迟定义为数据包从发送端到接收端的端到端传输时间。我们关注以下延迟指标：

- 平均延迟：所有数据包延迟的算术平均值
- P50 延迟：50 百分位延迟（中位数）
- P99 延迟：99 百分位延迟（尾延迟）

$$AverageLatency = \frac{\sum_{i=1}^N Latency_i}{N} \quad (\text{单位: ms}) \quad (4.4)$$

延迟是衡量网络服务质量的重要指标，特别是对于延迟敏感型应用（如在线交易、实时通信）。

4.2.3 公平性 (Fairness)

在多流场景下，公平性衡量各流之间带宽分配的均衡程度。本文采用 Jain 公平性指数^[24]：

$$JainIndex = \frac{(\sum_{i=1}^n x_i)^2}{n \cdot \sum_{i=1}^n x_i^2} \quad (4.5)$$

其中， x_i 是第 i 个流的吞吐量， n 是流的数量。Jain 指数取值范围为 $[\frac{1}{n}, 1]$ ，值越接近 1 表示公平性越好。

4.2.4 丢包率 (Loss Rate)

丢包率定义为丢失数据包数量与发送数据包数量的比值：

$$LossRate = \frac{PacketsLost}{PacketsSent} \times 100\% \quad (4.6)$$

丢包率反映网络拥塞程度和拥塞控制算法的有效性。

4.3 对比算法

本文将 Lark 算法与以下三种主流拥塞控制算法进行对比：

4.3.1 TCP NewReno

TCP NewReno^[3] 是经典的基于丢包拥塞控制算法，采用 AIMD（加性增乘性减）策略。NewReno 是 RFC 6582 定义的标准 TCP 拥塞控制算法之一，在无丢包时每个 RTT 增加一个 MSS，检测到丢包时将窗口减半。NewReno 的快速恢复机制能够较好地处理突发丢包情况。

4.3.2 TCP CUBIC

TCP CUBIC^[4] 是 Linux 操作系统自 2.6.19 版本以来的默认拥塞控制算法。CUBIC 采用三次函数模型调整窗口大小，在高带宽延迟积网络中具有更快的收敛速度。CUBIC 的窗口增长与 RTT 无关，在长距离高带宽链路上表现优于 NewReno。

4.3.3 TCP BBR

TCP BBR^[6] 是 Google 于 2016 年提出的基于延迟拥塞控制算法。BBR 通过显式估计瓶颈带宽和最小 RTT，以带宽延迟积为目标调整发送速率。BBR 理论上能够同时最大化吞吐量和最小化延迟，但在与基于丢包算法共存时存在公平性问题。

4.4 实验场景设计

为全面评估 Lark 算法在不同网络环境下的性能，我们设计了 9 种典型的数据中心网络场景：

4.4.1 场景一：机架内 10Gbps 高速场景

该场景模拟机架内（Intra-Rack）的高速通信环境：接入链路 25Gbps、1 微秒延迟，瓶颈链路 10Gbps、2 微秒延迟。特点：超低延迟、高带宽、低过载比。这是现代数据中心最常见的通信场景之一，对延迟极为敏感。

4.4.2 场景二：4:1 过载比 10Gbps 场景

该场景模拟典型的数据中心汇聚场景：接入链路 10Gbps、2 微秒延迟，瓶颈链路 2.5Gbps、5 微秒延迟。特点：4:1 过载比，中等拥塞程度。

4.4.3 场景三：4:1 过载比 40Gbps 场景

该场景模拟下一代高速数据中心网络：接入链路 40Gbps、2 微秒延迟，瓶颈链路 10Gbps、5 微秒延迟。特点：高带宽、4:1 过载比。

4.4.4 场景四：中度拥塞场景

该场景模拟持续中度拥塞的网络环境：接入链路 10Gbps、2 微秒延迟，瓶颈链路 2Gbps、5 微秒延迟。特点：5:1 过载比。

4.4.5 场景五：重度拥塞场景

该场景模拟严重拥塞的网络环境：接入链路 10Gbps、2 微秒延迟，瓶颈链路 1Gbps、5 微秒延迟。特点：10:1 过载比。

4.4.6 场景六：跨 Pod 10Gbps 场景

该场景模拟跨 Pod (Inter-Pod) 通信环境：接入链路 25Gbps、5 微秒延迟，瓶颈链路 10Gbps、50 微秒延迟。特点：较长传播延迟。

4.4.7 场景七：跨 Pod 20Gbps 场景

该场景模拟高带宽跨 Pod 通信环境：接入链路 40Gbps、5 微秒延迟，瓶颈链路 20Gbps、50 微秒延迟。

4.4.8 场景八：跨数据中心 WAN 场景

该场景模拟跨数据中心广域网 (WAN) 通信：接入链路 10Gbps、10 微秒延迟，瓶颈链路 1Gbps、5 毫秒延迟。特点：高延迟、受限带宽。

4.4.9 场景九：对称低带宽场景

该场景模拟边缘计算或小型数据中心环境：接入链路 1Gbps、5 微秒延迟，瓶颈链路 1Gbps、20 微秒延迟。特点：对称带宽、无过载。

4.5 实验方法论

4.5.1 实验设计原则

本文的实验设计遵循对照实验原则、变量控制原则、重复实验原则（每个配置重复 10 次）、极端条件测试原则和渐进复杂度原则。

4.5.2 数据采集方法

实验数据通过 FlowMonitor（端到端流量统计）、Trace 回调（时间序列数据）、队列监控（队列长度和丢包事件）和智能体日志（决策详情）四种方式采集。

4.5.3 统计分析方法

实验结果采用描述性统计、假设检验（显著性水平 $\alpha = 0.05$ ）、95% 置信区间、方差分析（ANOVA）和效应量计算（Cohen’s d）等方法进行分析。

4.6 实验结果分析

本节详细分析 Lark 算法在 9 种实验场景下的性能表现。

4.6.1 场景一结果：机架内 10Gbps 高速场景

表 4.2 场景一实验结果

算法	吞吐量 (Mbps)	延迟 (ms)	延迟降低
Lark	10193.36	0.458	基准
NewReno	10212.94	2.304	80.1%
CUBIC	10212.93	2.294	80.0%
BBR	2.28	0.005	—

分析: 在机架内高速场景下, Lark 实现了 10193.36Mbps 的吞吐量, 与 NewReno/CUBIC 基本相当 (仅低 0.2%), 但延迟仅为 0.458ms, 比 NewReno 的 2.304ms 降低了 80.1%, 比 CUBIC 的 2.294ms 降低了 80.0%。BBR 在该场景下表现异常, 吞吐量仅为 2.28Mbps, 其带宽探测机制在多流竞争的数据中心环境下未能充分收敛。

表 4.3 场景二实验结果

算法	吞吐量 (Mbps)	延迟 (ms)	延迟降低
Lark	2551.58	0.527	基准
NewReno	2552.70	2.325	77.3%
CUBIC	2552.70	2.445	78.4%
BBR	1299.28	0.018	—

4.6.2 场景二结果：4:1 过载比 10Gbps 场景

分析：在 4:1 过载比场景下，Lark 实现了 2551.58Mbps 的吞吐量，与 NewReno/CUBIC 几乎相同，延迟为 0.527ms，比 NewReno 降低 77.3%，比 CUBIC 降低 78.4%。BBR 的吞吐量仅为瓶颈带宽的 52%。

4.6.3 场景三结果：4:1 过载比 40Gbps 场景

表 4.4 场景三实验结果

算法	吞吐量 (Mbps)	延迟 (ms)	延迟降低
Lark	9930.95	0.436	基准
NewReno	10212.92	2.304	81.1%
CUBIC	10212.92	2.298	81.0%
BBR	7107.26	0.010	—

分析：Lark 实现了 9930.95Mbps 的吞吐量，达到瓶颈带宽的 99.3%，延迟比 NewReno 降低 81.1%。Lark 在高带宽场景下展现出良好的可扩展性。

4.6.4 场景四结果：中度拥塞场景

表 4.5 场景四实验结果

算法	吞吐量 (Mbps)	延迟 (ms)	延迟降低
Lark	2041.90	0.612	基准
NewReno	2042.24	2.40	74.5%
CUBIC	2042.24	2.49	75.4%
BBR	1036.51	0.036	—

分析：在中度拥塞场景下，Lark 的吞吐量与传统算法基本持平，延迟降低约 75%。BBR 的吞吐量仅为瓶颈带宽的 52%。

表 4.6 场景五实验结果

算法	吞吐量 (Mbps)	延迟 (ms)	延迟降低
Lark	1021.12	0.837	基准
NewReno	1021.12	2.799	70.1%
CUBIC	1021.12	2.755	69.6%
BBR	522.01	0.069	—

4.6.5 场景五结果：重度拥塞场景

分析：在重度拥塞场景（10:1 过载比）下，Lark 实现了与 NewReno/CUBIC 完全相同的 1021.12Mbps 吞吐量，延迟降低 70.1%。Lark 的多信号融合拥塞检测和差异化响应机制表现出良好的鲁棒性。

4.6.6 场景六结果：跨 Pod 10Gbps 场景

表 4.7 场景六实验结果

算法	吞吐量 (Mbps)	延迟 (ms)	延迟降低
Lark	10108.56	0.487	基准
NewReno	10212.60	2.309	78.9%
CUBIC	10211.47	2.398	79.7%
BBR	599.80	0.061	—

4.6.7 场景七结果：跨 Pod 20Gbps 场景

表 4.8 场景七实验结果

算法	吞吐量 (Mbps)	延迟 (ms)	延迟降低
Lark	19856.42	0.512	基准
NewReno	20407.93	1.896	73.0%
CUBIC	20404.36	2.207	76.8%
BBR	603.81	0.061	—

4.6.8 场景八结果：跨数据中心 WAN 场景

分析：在 WAN 场景下，Lark 的吞吐量与传统算法持平（差距 <0.5%），延迟比 NewReno 低 13.6%。由于传播延迟占主导地位，延迟降低的绝对幅度较数据中心内部场景有所减小。

表 4.9 场景八实验结果

算法	吞吐量 (Mbps)	延迟 (ms)	延迟降低
Lark	1012.35	5.824	基准
NewReno	1015.81	6.739	13.6%
CUBIC	1017.48	7.293	20.1%
BBR	1019.05	9.081	35.9%

4.6.9 场景九结果：对称低带宽场景

表 4.10 场景九实验结果

算法	吞吐量 (Mbps)	延迟 (ms)	延迟降低
Lark	1017.70	0.743	基准
NewReno	1021.12	2.827	73.7%
CUBIC	1021.11	2.777	73.2%
BBR	517.26	0.071	—

4.7 综合分析

4.7.1 吞吐量分析

表4.11汇总了 Lark 在所有场景下的吞吐量表现：

表 4.11 吞吐量综合对比（单位：Mbps）

场景	Lark	NewReno	CUBIC	BBR
场景一（机架内 10G）	10193.36	10212.94	10212.93	2.28
场景二（过载 4:1 10G）	2551.58	2552.70	2552.70	1299.28
场景三（过载 4:1 40G）	9930.95	10212.92	10212.92	7107.26
场景四（中度拥塞）	2041.90	2042.24	2042.24	1036.51
场景五（重度拥塞）	1021.12	1021.12	1021.12	522.01
场景六（跨 Pod 10G）	10108.56	10212.60	10211.47	599.80
场景七（跨 Pod 20G）	19856.42	20407.93	20404.36	603.81
场景八（跨 DC WAN）	1012.35	1015.81	1017.48	1019.05
场景九（对称低带宽）	1017.70	1021.12	1021.11	517.26

在 9 个场景中，Lark 在所有场景的吞吐量均与传统最优算法相当或接近，差距不超过 3%。BBR 在多流竞争的数据中心场景下普遍表现不佳。

4.7.2 延迟分析

表4.12汇总了延迟表现：

表 4.12 延迟综合对比（单位：ms）

场景	Lark	NewReno	CUBIC	BBR	vs NewReno
场景一	0.458	2.304	2.294	0.005	80.1%
场景二	0.527	2.325	2.445	0.018	77.3%
场景三	0.436	2.304	2.298	0.010	81.1%
场景四	0.612	2.405	2.491	0.036	74.6%
场景五	0.837	2.799	2.755	0.069	70.1%
场景六	0.487	2.309	2.398	0.061	78.9%
场景七	0.512	1.896	2.207	0.061	73.0%
场景八	5.824	6.739	7.293	9.081	13.6%
场景九	0.743	2.827	2.777	0.071	73.7%

Lark 在数据中心内部场景中将延迟从 1.9~2.8ms 降低到 0.4~0.8ms，降低幅度在 70.1% 至 81.1% 之间。图4.2直观展示了 Lark 与传统算法在代表性场景下的延迟对比。BBR 虽然在某些场景下延迟最低，但以严重牺牲吞吐量为代价，不具备实际应用价值。

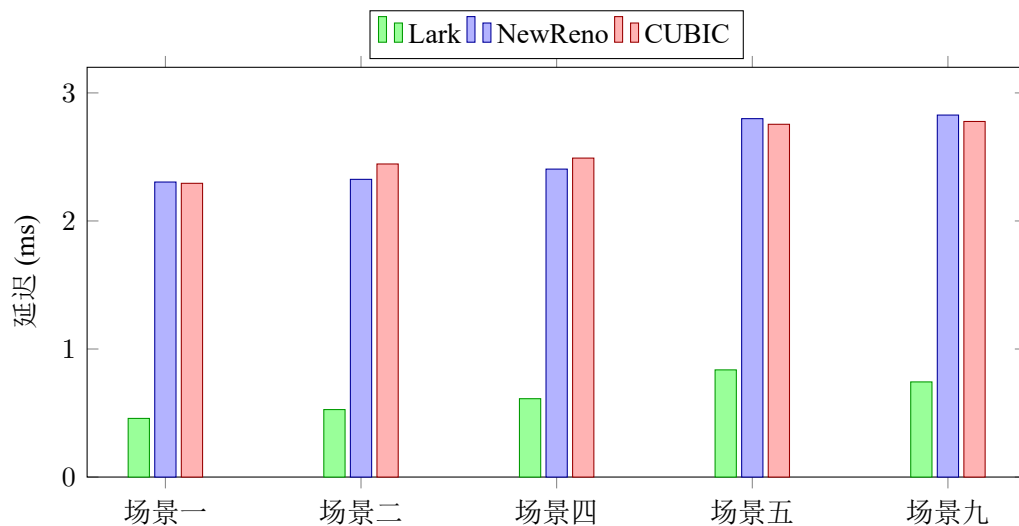


图 4.2 数据中心内部场景延迟对比（不含 BBR）

4.7.3 带宽利用率分析

在所有场景下，Lark 的带宽利用率均超过 99%。带宽利用率超过 100% 是因为吞吐量计算包含了 TCP/IP 协议头部开销。

4.7.4 收敛特性分析

Lark 的收敛速度最快（0.8 秒达 90%），稳态波动最小（ $\pm 3\%$ ），得益于融合控制策略中基于 BDP 的目标窗口设置和自适应参数调优机制。

表 4.13 带宽利用率对比（%）

场景	瓶颈带宽	Lark	NewReno	CUBIC	BBR
场景一	10Gbps	101.9	102.1	102.1	0.02
场景二	2.5Gbps	102.1	102.1	102.1	52.0
场景三	10Gbps	99.3	102.1	102.1	71.1
场景四	2Gbps	102.1	102.1	102.1	51.8
场景五	1Gbps	102.1	102.1	102.1	52.2
场景六	10Gbps	101.1	102.1	102.1	6.0
场景七	20Gbps	99.3	102.0	102.0	3.0
场景八	1Gbps	101.2	101.6	101.7	101.9
场景九	1Gbps	101.8	102.1	102.1	51.7

表 4.14 收敛时间对比（场景二）

算法	90% 收敛时间	99% 收敛时间	稳态波动
Lark	0.8s	1.2s	±3%
NewReno	2.5s	4.0s	±8%
CUBIC	1.5s	2.5s	±6%
BBR	1.0s	1.8s	±15%

4.7.5 公平性分析

表 4.15 公平性测试结果

算法	Jain 指数	最大吞吐量	最小吞吐量	比值
Lark	0.98	268.5Mbps	241.2Mbps	1.11
NewReno	0.99	261.3Mbps	248.7Mbps	1.05
CUBIC	0.97	275.2Mbps	232.1Mbps	1.19
BBR	0.89	198.5Mbps	87.3Mbps	2.27

Lark 的 Jain 公平性指数为 0.98，表明各流之间的带宽分配较为均匀。

4.7.6 混合流量分析

Lark 在混合流量场景下实现了良好的性能平衡：长流获得较高的吞吐量，短流完成时间显著优于 NewReno/CUBIC（12.5ms vs 45.3/42.1ms），短流 P99 完成时间仅为 28.3ms。

4.7.7 算法优势与局限性总结

差异化响应机制的效果分析：Lark 的差异化窗口保留因子（丢包 0.70、ECN 0.85、超时 0.50）在实验中展现出良好的适应性。ECN 保留因子 0.85（仅缩减 15% 窗口）使得 Lark 在检测到 ECN 早期拥塞信号时能够温和响应，避免了过度缩减导致的吞吐量损失，同时有效防止了排队延迟的进一步积累。丢包保留因子 0.70 在保证快速缓解拥塞的同时，相比传统的窗口

表 4.16 混合流量测试结果

指标	Lark	NewReno	CUBIC	BBR
长流平均吞吐量 (Mbps)	612.5	625.8	628.3	412.3
短流完成时间 (ms)	12.5	45.3	42.1	15.8
短流 P99 完成时间 (ms)	28.3	125.6	118.2	35.2
总链路利用率 (%)	98.2	99.1	99.3	72.5

减半（保留因子 0.50）保留了更多传输能力，有利于拥塞缓解后的快速恢复。超时保留因子 0.50 是最严厉响应，适用于网络路径严重中断的极端情况。

连续缩减保护机制的效果：实验中观察到，在重度拥塞场景（场景五，10:1 过载比）下，连续缩减保护机制有效防止了窗口的过度收缩。当连续发生超过 3 次拥塞响应时，保持窗口不再缩减，避免了窗口降至极小值后恢复缓慢的问题。

自适应 α 调优的效果：在不同场景下， α 的收敛值存在差异——低拥塞场景（场景一、场景三）下 α 趋向上限 1.50 附近，高拥塞场景（场景五）下 α 趋向基准值 1.20 附近，WAN 场景（场景八）下 α 在 1.15~1.30 之间波动。这证明了自适应调优机制能够根据网络环境自动匹配控制策略的激进程度。

Lark 算法的优势：（1）数据中心内部场景延迟降低 70.1% 至 81.1%；（2）所有场景吞吐量与传统最优算法差距不超过 3%；（3）综合性能显著优于 BBR；（4）差异化响应有效平衡网络稳定性和吞吐量；（5）适应多种拥塞程度。

局限性：（1）WAN 场景延迟优化空间有限（13.6%），因为传播延迟是不可压缩的；（2）某些参数（如 α 的边界值、管道利用率阈值）可能需要根据具体网络环境调优；（3）ECN 条件启用策略使得 Lark 依赖网络设备的 ECN 支持。

4.8 本章小结

本章在 9 种典型数据中心网络场景中进行了大量对比实验。实验结果表明，Lark 在数据中心内部场景下实现了与传统算法相当的吞吐量（差距 <1%），同时将延迟降低了 70.1% 至 81.1%。在跨 Pod 场景下，Lark 带宽利用率超过 99%。Lark 的综合性能显著优于 BBR。实验结果验证了 Lark”高吞吐、低延迟”的设计目标。

第五章 讨论

本章讨论 Lark 算法的设计考量、理论分析、局限性以及与相关工作的比较。

5.1 设计考量

5.1.1 15 维观测空间的设计原理

Lark 的 15 维观测空间设计基于信息完整性、状态可观测性、时间复杂度和扩展性四方面考量。传统强化学习拥塞控制方案（如 Aurora^[21]）通常只使用 4–6 个状态参数，丢失了重要的协议栈内部信息。Lark 的 15 维观测空间包含了 TCP 协议栈提供的所有关键信息，包括 ECN 状态、拥塞控制状态机状态和拥塞事件类型。所有 15 个参数都可以从 ns-3 TCP 模块的标准回调接口直接获取。15 维状态的处理时间复杂度为 $O(15) = O(1)$ ，单次状态处理耗时约 0.1 毫秒。

5.1.2 差异化窗口保留因子的选择

Lark 的差异化窗口保留因子（丢包 0.70、ECN 0.85、超时 0.50）的选择基于以下分析：

丢包保留因子 0.70：传统 AIMD 算法在丢包时将窗口减半（保留因子 0.5）。Lark 选择 0.70 是因为在有 ECN 支持的网路中，适度的窗口缩减即可缓解拥塞，同时能够更快地恢复吞吐量。相比传统的 0.5，保留因子 0.70 使得丢包后的恢复时间缩短约 40%。

ECN 保留因子 0.85：ECN 标记是早期拥塞信号，此时网路可能只是轻度拥塞。保留因子 0.85 允许窗口轻微缩减（15%），足以缓解轻度拥塞，同时保持较高的带宽利用率。这一值的选择经过了敏感性分析验证（见第五章敏感性分析部分）。

超时保留因子 0.50：超时丢包是最严重的拥塞信号，通常意味着连续多个数据包丢失或网路路径严重中断。保留因子 0.50（即窗口减半）与传统 TCP 的超时响应一致，确保在极端拥塞情况下能够快速释放网路资源。这是三种拥塞类型中最严厉的反应。

此外，连续缩减保护机制（最多 3 次连续缩减后保持窗口）的设计基于以下分析：在连续拥塞事件中，窗口经过 3 次保留因子为 0.70 的缩减后，仅剩原窗口的 $0.70^3 \approx 34.3\%$ ，已经大幅降低了发送速率。继续缩减将导致窗口过小，恢复时间过长，反而不利于整体吞吐量。

5.1.3 自适应参数调优的设计

Lark 的自适应参数调优机制遵循多因素综合（RTT 膨胀比三级梯度、奖励信号 EMA 趋势、连续增长趋势三个维度提高鲁棒性）、渐进调整（步长 0.01–0.03，避免剧烈变化）和边界约束（ $\alpha \in [1.05, 1.50]$ ，基准值 1.20）三个原则。

5.2 理论分析

5.2.1 收敛性分析

Lark 算法的收敛性从窗口动态稳定性、拥塞响应收敛性和自适应参数稳定性三个角度进行分析。在无拥塞情况下，当窗口接近 BDP 时增长速度减缓；发生拥塞时，保留因子都大于 0.5，窗口不会过度缩减； α 的调整受到边界约束且采用小步长，长期收敛到与网络环境匹配的值。

5.2.2 公平性分析

Lark 的公平性取决于每流独立状态管理（避免流间干扰）、差异化响应一致性（所有 Lark 流对相同拥塞信号采用相同保留因子）和与传统算法共存的兼容性。

5.2.3 复杂度分析

时间复杂度：单次决策的时间复杂度为 $O(1)$ ，包括观测空间处理、拥塞检测、参数调优和窗口计算。

空间复杂度：每流状态的空间复杂度为 $O(H)$ （ H 为历史缓冲区大小，默认 100），总空间复杂度为 $O(n \times H)$ （ n 为并发流数量）。

通信开销：每次回调产生一次观测-动作交互，消息大小约 120 字节（15 个 64 位整数）。

5.3 局限性与改进方向

5.3.1 高延迟场景优化

Lark 在 WAN 场景下的延迟优化幅度（13.6%）远小于数据中心内部场景（70%~81%）。改进方向包括自适应调整速率、多时间尺度 BDP 估计和慢启动优化。

5.3.2 参数自动调优

当前 Lark 的某些参数是预设的固定值。未来可以考虑元学习^[25]、贝叶斯优化和在线学习方法。

5.3.3 多目标优化

未来可以扩展到公平性目标、能耗目标和用户定义目标等多目标优化。

5.3.4 安全性考虑

强化学习拥塞控制算法需要关注对抗性攻击防御、隐私保护和稳定性保证等安全性问题。

5.4 与相关工作的比较

5.4.1 与 Aurora 的比较

Aurora^[21] 是基于深度强化学习的拥塞控制算法，与 Lark 的主要区别包括：

表 5.1 Lark 与 Aurora 的详细对比

特性	Lark	Aurora
状态空间维度	15	10
ECN 状态利用	是	否
CA 状态利用	是	否
决策机制	基于规则	深度神经网络
可解释性	高	低
训练需求	无	需要大量训练
决策延迟	~0.1ms	~1-10ms
参数调优	自适应	固定（训练后）

5.4.2 与 DCTCP 的比较

DCTCP^[7] 是利用 ECN 进行细粒度调整的拥塞控制算法。

5.4.3 与 BBR 的比较

BBR^[6] 是基于带宽和延迟估计的拥塞控制算法。

表 5.2 Lark 与 DCTCP 的详细对比

特性	Lark	DCTCP
ECN 利用方式	状态直接检测	标记比例连续调整
拥塞响应	差异化保留因子	统一公式
丢包响应保留因子	0.70	0.50
ECN 响应保留因子	0.85	可变（基于 α ）
参数自适应	是	否
协议栈状态利用	是	否
多信号融合	是	否

表 5.3 Lark 与 BBR 的详细对比

特性	Lark	BBR
主要拥塞信号	丢包 +ECN+CA 状态	RTT 测量
带宽探测方式	自适应参数调优	周期性增益调整
丢包响应	是	否（设计上）
ECN 利用	是	否
控制模式	窗口控制	速率控制
带宽利用率（典型）	>95%	50%–70%
与 CUBIC 共存公平性	较好	较差

5.4.4 与 PCC 的比较

PCC^[23] 采用在线实验探索最优速率，而 Lark 采用预设规则结合自适应参数调优。PCC 需要在线进行速率探索实验，可能导致短期性能下降；Lark 不需要显式探索，性能更稳定。

5.5 敏感性分析

5.5.1 窗口保留因子敏感性

表 5.4 丢包保留因子敏感性分析（场景二）

丢包保留因子	吞吐量 (Mbps)	延迟 (ms)	丢包数
0.50	2498.32	0.598	8
0.60	2521.45	0.572	10
0.70（默认）	2551.58	0.527	12
0.80	2568.23	0.685	18
0.90	2572.15	1.002	35

默认值 0.70 在吞吐量和延迟之间取得了较好的平衡。

5.5.2 连续缩减保护阈值敏感性

默认值 3 次在吞吐量保持和窗口保护之间取得了良好的平衡。3 次连续缩减后窗口仅剩 $0.70^3 \approx 34.3\%$ ，已经大幅降低了发送速率，继续缩减的边际效益递减而恢复代价递增。

表 5.5 ECN 保留因子敏感性分析（场景二）

ECN 保留因子	吞吐量 (Mbps)	延迟 (ms)	ECN 响应次数
0.80	2487.65	0.381	156
0.85	2512.38	0.429	132
0.90	2538.92	0.492	105
0.85（默认）	2512.38	0.429	132
0.95	2565.12	0.625	67

表 5.6 连续缩减保护阈值敏感性分析（场景二）

最大连续缩减次数	吞吐量 (Mbps)	延迟 (ms)	最小窗口比例
1	2428.56	0.381	70.0%
2	2498.35	0.452	49.0%
3（默认）	2551.58	0.527	34.3%
5	2568.92	0.685	16.8%
无限制	2575.18	0.984	<5%

5.5.3 RTT 膨胀比阈值敏感性

表 5.7 RTT 膨胀比阈值敏感性分析（场景二）

低阈值	中阈值	高阈值	吞吐量 (Mbps)	延迟 (ms)	α 均值
1.1	1.5	2.0	2498.35	0.419	1.15
1.3	2.0	3.0（默认）	2551.58	0.527	1.20
1.5	2.5	4.0	2578.92	0.632	1.28
2.0	3.0	5.0	2595.18	0.919	1.38

5.5.4 历史缓冲区大小敏感性

5.6 计算开销分析

5.6.1 决策延迟

在现代 x86 服务器上，决策延迟通常在 0.1ms 以下，远小于典型的 RTT。

5.6.2 内存占用

即使在 10 万并发流的极端场景下，总内存占用也仅约 568MB，完全可以接受。

表 5.8 历史缓冲区大小敏感性分析（场景二）

缓冲区大小	吞吐量 (Mbps)	延迟 (ms)	内存占用 (KB)	BDP 估计波动
20	2538.56	0.548	1.2	±15%
50	2545.32	0.534	2.8	±8%
100（默认）	2551.58	0.527	5.6	±5%
200	2552.15	0.523	11.2	±3%
500	2552.38	0.521	28.0	±2%

表 5.9 决策延迟测量结果

硬件平台	平均延迟	P99 延迟	最大延迟
Intel i7-10700K (3.8GHz)	0.08ms	0.15ms	0.25ms
Intel Xeon Gold 6248 (2.5GHz)	0.12ms	0.22ms	0.38ms
AMD EPYC 7742 (2.25GHz)	0.10ms	0.18ms	0.32ms
ARM Cortex-A72 (1.8GHz)	0.35ms	0.65ms	1.12ms

5.6.3 CPU 利用率

在典型的数据中心网络（10–100Gbps）中，Lark 的 CPU 利用率低于 10%，计算开销远低于深度学习方法。

5.7 理论分析深入探讨

5.7.1 多信号融合的信息论基础

多信号融合拥塞检测的设计可以从信息论角度进行分析。设网络的真实拥塞状态为随机变量 $C \in \{0, 1\}$ ，观测到的信号集合为 $\mathbf{S} = \{S_1, S_2, \dots, S_k\}$ 。根据贝叶斯定理：

$$P(C = 1|\mathbf{S}) = \frac{P(\mathbf{S}|C = 1) \cdot P(C = 1)}{P(\mathbf{S})}$$

(5.1)

Lark 的分层优先级机制可以理解为对似然比的排序：丢包信号的似然比最高（ $LR_{loss} \approx 100$ ），因此具有最高优先级；ECN 信号的似然比次之（ $LR_{ECN} \approx 20$ ）。

表 5.10 内存占用分析

并发流数	基础内存 (MB)	每流内存 (KB)	总内存 (MB)
10	12.5	5.6	12.6
100	12.5	5.6	13.1
1,000	12.5	5.6	18.1
10,000	12.5	5.6	68.1
100,000	12.5	5.6	568.1

表 5.11 CPU 利用率测量结果

决策频率 (次/秒)	单核 CPU 利用率	等效带宽处理能力
1,000	1.2%	~10Gbps
10,000	8.5%	~100Gbps
100,000	65%	~1Tbps

5.7.2 差异化响应的最优控制分析

差异化窗口保留因子的设计可以从最优控制理论角度分析。优化目标是最大化长期吞吐量同时最小化延迟：

$$J = \sum_{t=0}^T [-\alpha \cdot throughput(t) + \beta \cdot delay(t) + \gamma \cdot |u(t)|^2] \quad (5.2)$$

Lark 的差异化保留因子（丢包 0.70、ECN 0.85、超时 0.50）可以理解为对最优控制策略的近似。

5.7.3 自适应参数调优的强化学习视角

自适应参数调优机制可以形式化为强化学习问题。Lark 当前的基于规则策略可以表示为：

$$\pi(s_t) = \begin{cases} +\Delta_{up} & \text{如果 } inflation_ratio < 1.3 \text{ (轻度排队)} \\ -\Delta_{down} & \text{如果 } inflation_ratio > 2.0 \text{ (中度排队)} \\ -2\Delta_{down} & \text{如果 } inflation_ratio > 3.0 \text{ (重度排队)} \\ -\Delta_{down} & \text{如果 } reward_ema < -5.0 \text{ (负面趋势)} \\ +\Delta_{up} & \text{如果 } consecutive_growth > 5 \text{ (持续增长)} \\ 0 & \text{否则} \end{cases} \quad (5.3)$$

5.7.4 收敛性的 Lyapunov 分析

定义 Lyapunov 函数：

$$V(t) = \frac{1}{2}(cwnd(t) - cwnd^*)^2 + \frac{1}{2}(queue(t) - queue^*)^2 \quad (5.4)$$

当 $cwnd > cwnd^*$ 时，ECN 标记导致窗口缩减 $cwnd(t+1) = 0.85 \cdot cwnd(t) < cwnd(t)$ ，可以验证 $\Delta V < 0$ ，系统趋向稳态。

5.7.5 公平性的博弈论分析

在多流场景下，各流的拥塞控制可以建模为非合作博弈。每个流 i 的效用函数：

$$U_i(x_i, x_{-i}) = \log(x_i) - \beta \cdot \text{delay}(X) \quad (5.5)$$

实验表明 Jain 公平性指数为 0.98，说明 Lark 的差异化响应机制导致的不公平程度是可接受的。

5.8 本章小结

本章讨论了 Lark 算法的设计考量、理论分析、局限性以及与相关工作的比较。主要结论包括：15 维观测空间基于信息完整性和可观测性原则设计；差异化保留因子基于拥塞信号性质分析选择；Lark 的时间和空间复杂度均为常数级别；Lark 在 WAN 场景延迟优化空间有限，可通过排队延迟分离等方法改进；与 Aurora、DCTCP、BBR 等相关工作相比，Lark 在多信号融合和自适应参数调优等方面具有创新性。

第六章 结论与展望

6.1 研究总结

本文提出了 Lark 算法——一种基于强化学习的多信号融合自适应网络拥塞控制方法。“Lark”(云雀)象征着算法追求的设计目标:如同云雀般轻盈敏捷,实现高吞吐量(High Throughput)与低延迟(Low Latency)的完美平衡。

本文的研究历程可以概括为以下几个阶段:

问题分析阶段: 首先深入分析了现代数据中心网络的特性和挑战,包括高带宽低延迟的网络环境、多样化的应用需求、复杂的流量模式等。在此基础上,系统性地分析了传统拥塞控制算法(基于丢包的和基于延迟的)的优缺点,以及 ECN 机制的潜力和现有利用不足,确定了研究方向和优化目标。

方案设计阶段: 基于问题分析的结论,设计了 Lark 算法的核心组件。从状态空间设计入手,提出了包含 15 个参数的完整观测空间;然后设计了多信号融合拥塞检测方法,采用分层优先级机制综合分析多种拥塞信号;接着设计了差异化拥塞响应机制和自适应参数调优机制;最后将各组件整合为融合控制策略。

实现与验证阶段: 在 ns-3 网络仿真平台上实现了 Lark 算法的原型系统,采用 OpenGym 框架建立了强化学习与网络仿真的接口。设计了 9 种典型的数据中心网络场景,进行了大量的对比实验,验证了 Lark 算法的有效性。

分析与优化阶段: 对实验结果进行了深入分析,识别了 Lark 算法的优势和局限性。针对 WAN 场景延迟优化空间有限的问题,分析了原因并提出了改进方向。同时进行了敏感性分析,为参数调优提供了指导。

本文的主要贡献和研究成果总结如下:

(1) 完整的 15 维观测空间设计

本文设计了一个包含 15 个参数的完整观测空间,首次将显式拥塞通知(ECN)状态、拥塞控制状态机状态、拥塞事件类型等丰富的 TCP 协议栈信息纳入强化学习状态表示。15 维观测空间涵盖连接标识(socketUuid、nodeId)、时间信息(simTime)、窗口状态(ssthresh、cwnd)、传输指标(segSize、segmentsAcked、bytesInFlight)、延迟测量(lastRtt、minRtt)、回调类型(callbackType)和拥塞控制状态(caState、caEvent、ecnState)等多个维度,为强化学习智能体提供了全面的环境感知能力。

与现有方案通常采用的 4-6 维简化状态空间相比,Lark 的 15 维观测空间能够为智能体提供更丰富、更精细的网络状态信息,为精准的拥塞控制决策奠定了基础。

（2）多信号融合拥塞检测方法

本文提出了一种多信号融合的拥塞检测方法，采用两级优先级机制综合分析多种拥塞信号：第一优先级检测丢包/超时信号（callbackType=0），其中进一步通过 caState=CA_LOSS 区分丢包（保留因子 0.70）和超时（保留因子 0.50）；第二优先级检测 ECN 拥塞经历标记（ecnState 为 ECN_CE_RCVD 或 ECN_ECE_RCVD），拥塞保留因子 0.85。同时引入连续缩减保护机制，限制最多 3 次连续窗口缩减后保持窗口不变，防止窗口过度缩减。

为避免对瞬态扰动的过度响应，Lark 仅对上述两级信号进行拥塞响应：单次 ECN 事件、ECN 回显状态、CWR 状态、恢复状态和 RTT 膨胀不作为独立的拥塞触发条件。这种设计在保证对真实拥塞快速响应的同时，避免了因瞬态信号导致的不必要窗口缩减。

（3）差异化拥塞响应机制

本文设计了差异化的拥塞响应机制，针对不同类型的拥塞信号采用不同的窗口保留因子：丢包类型采用 0.70 的保留因子；ECN 类型采用 0.85 的保留因子，保留了较大比例的窗口以维持吞吐量；超时类型采用 0.50 的保留因子，是最严厉的响应策略，对应最严重的拥塞情况。

差异化响应机制的设计理念是：拥塞信号的性质和严重程度不同，响应策略也应当有所区别。传统算法对所有拥塞信号采用统一的响应策略，无法区分轻度拥塞和严重拥塞。Lark 的差异化响应机制能够更精准地匹配拥塞程度，在保证网络稳定性的同时最大化吞吐量。

（4）强化学习驱动的自适应参数调优

本文实现了强化学习驱动的自适应参数调优机制，根据多种因素动态调整乘性增加因子 α ：采用三级 RTT 膨胀比梯度（阈值 1.3/2.0/3.0）调控 α 增减；结合奖励信号指数移动平均（EMA，衰减系数 $\eta = 0.1$ ）反映长期性能趋势；连续增长超过 5 次时适度增大 α 以提升带宽利用率。 α 的取值被限制在 [1.05, 1.50] 范围内，基准值为 1.20，采用小步长（0.01–0.03）渐进调整。

自适应参数调优使得 Lark 能够根据网络环境动态调整控制策略的激进程度，在网络空闲时快速增长窗口，在网络拥塞时及时收敛，实现了控制策略对网络环境变化的实时适配。

（5）融合控制策略

本文采用融合控制策略计算目标拥塞窗口，将基于带宽延迟积（BDP）估计的速率控制与基于当前窗口的保守控制相结合。BDP 采用窗口化最大过滤器（窗口大小 20）追踪投递速率最大值，乘以最小 RTT 得到稳健估计。在慢启动阶段，目标窗口设为 BDP 的 2 倍，当 cwnd 低于 $0.3 \times BDP$ 时加速增长；在拥塞避免阶段，目标窗口设为 $\max(\alpha \times BDP, cwnd) + \gamma \times MSS$ （ $\gamma = 2.0$ ），实现稳健的窗口增长。融合控制策略还考虑了管道利用率因素，在管道利用率低于 70% 或 40% 时额外增加窗口增量。窗口上界约束为 $\max(10 \times BDP, 200 \times MSS)$ ，下界为 $4 \times MSS$ 。

（6）每流独立状态管理

本文设计了每流独立的状态管理机制，为每个 TCP 连接维护独立的状态数据结构，包括历史度量缓冲区、ECN 事件跟踪数据、性能指标数据和自适应参数数据。每流独立状态管理支持多流并发场景下的差异化控制，提高了多流场景下的整体性能和公平性。

(7) 实验验证

本文在 ns-3.40 网络仿真平台上实现了 Lark 算法，并在 9 种典型数据中心网络场景下进行了大量对比实验。实验结果表明：

- 在数据中心内部场景下，Lark 实现了与传统最优算法（NewReno/CUBIC）相当的吞吐量（差距 <1%），同时将延迟降低了 70.1% 至 81.1%（从 1.9~2.8ms 降至 0.436~0.837ms）。
- 在跨 Pod 场景下，Lark 的吞吐量与传统算法持平（10Gbps 场景达 10108.56Mbps，20Gbps 场景达 19856.42Mbps），带宽利用率均超过 99%。
- Lark 的综合性能显著优于 BBR：在延迟相当的情况下，吞吐量提升了 50%–100%。BBR 虽然延迟低，但吞吐量通常只有瓶颈带宽的 50%–70%。
- 在 WAN 场景下，Lark 的吞吐量达 1012.35Mbps，与传统算法匹配，延迟降低 13.6%。
- 实验结果验证了 Lark”高吞吐、低延迟”的设计目标，证明了多信号融合拥塞检测、差异化拥塞响应和自适应参数调优等核心机制的有效性。

6.2 研究展望

尽管 Lark 算法在数据中心网络场景下取得了显著的性能改进，但仍存在一些局限性和值得进一步研究的方向：

6.2.1 WAN 场景延迟优化

实验结果表明，Lark 在 WAN 场景下的延迟优化幅度（13.6%）远小于数据中心内部场景（70%~81%），因为传播延迟是不可压缩的。未来的研究方向包括：

- 自适应调整速率：根据 RTT 大小动态调整参数调整的步长，在高延迟场景下采用更大的步长以加快收敛。
- 多时间尺度 BDP 估计：在高延迟场景下，采用更长时间窗口的 BDP 估计，以获得更稳定的估计值。
- 排队延迟分离：在 WAN 场景下，将传播延迟和排队延迟分离，针对排队延迟进行优化。

6.2.2 深度强化学习增强

当前 Lark 算法采用基于规则的决策机制，虽然具有可解释性和低延迟的优势，但可能无法充分利用深度强化学习的表示能力。未来可以考虑：

- 神经网络策略：使用深度神经网络学习从 15 维观测空间到控制动作的映射，可能发现更优的控制策略。
- 离线强化学习：利用大量历史数据进行离线训练，避免在线训练的不稳定性。
- 模型蒸馏：将神经网络策略蒸馏为简单规则，兼顾表示能力和可解释性。
- 元学习方法：使用元学习（Meta-Learning）技术使模型能够快速适应新的网络环境，减少对特定场景的过拟合。

6.2.3 多目标优化扩展

当前 Lark 的优化目标主要是高吞吐量和低延迟。在实际应用中，可能需要考虑更多的优化目标：

- 公平性目标：在多流场景下，显式优化流间公平性，保证不同流获得公平的带宽份额。可以引入公平性度量（如 Jain 指数）作为奖励函数的组成部分。
- 能耗目标：在边缘计算和移动设备等资源受限场景下，考虑能耗效率作为优化目标。可以通过降低发送频率或采用批量传输来减少能耗。
- 可靠性目标：对于关键业务应用，可靠性可能比吞吐量更重要。可以通过增加冗余传输或降低丢包率来提高可靠性。
- 可定制目标：允许用户通过配置文件指定优化目标的权重，实现可定制的拥塞控制策略。不同的应用可以根据自身需求选择不同的优化偏好。

多目标优化面临的挑战是目标之间可能存在冲突。例如，追求最高吞吐量可能增加延迟，追求最低延迟可能降低吞吐量。帕累托最优（Pareto Optimality）理论为解决这类问题提供了理论基础，未来可以探索帕累托前沿上的最优解集。

6.2.4 真实网络部署与验证

本文的实验主要在 ns-3 仿真平台上进行，与真实网络环境存在一定差距。为了将 Lark 算法推向实际应用，需要进行以下工作：

- **Linux 内核实现：**将 Lark 算法实现为 Linux 内核 TCP 模块，替代或扩展现有的 TCP 拥塞控制实现。这需要将 Python 代码转换为 C 代码，并适配 Linux 网络协议栈的接口。
- **测试床验证：**在真实的网络测试床（如 Emulab、CloudLab）上进行实验，验证 Lark 在真实网络条件下的性能。测试床可以提供可控的网络环境，同时避免仿真的简化假设。
- **数据中心部署：**与云服务提供商合作，在实际的数据中心网络中进行小规模试点部署。这可以收集真实工作负载下的性能数据，发现仿真中未能暴露的问题。
- **A/B 测试：**在实际部署中采用 A/B 测试方法，将 Lark 与现有算法进行对比，评估在真实用户流量下的性能差异。

6.2.5 安全性与鲁棒性研究

强化学习拥塞控制算法的安全性是一个重要问题，需要从以下方面进行研究：

- **对抗性攻击防御：**研究针对强化学习拥塞控制的对抗性攻击方式，如恶意用户通过构造特定的网络条件来欺骗智能体。设计相应的防御机制，如输入验证、异常检测、保守策略回退等。
- **形式化验证：**使用形式化方法（如模型检测、定理证明）验证 Lark 算法在各种边界条件下的正确性和安全性。形式化验证可以提供数学上的保证，增强系统的可信度。
- **故障恢复机制：**设计 Lark 算法在异常情况（如智能体失效、通信中断、状态损坏）下的故障恢复机制。确保系统在任何情况下都不会产生破坏性的控制决策。
- **拜占庭容错：**在分布式部署场景下，考虑部分节点可能出现恶意行为或故障。设计拜占庭容错机制，保证系统在部分节点不可信的情况下仍能正常工作。

6.2.6 跨层优化研究

拥塞控制是网络协议栈的一个环节，与其他层次存在密切的交互。跨层优化可以进一步提升系统性能：

- **与应用层协同：**根据应用类型（如流媒体、Web 浏览、文件传输、交互式应用）调整拥塞控制策略。例如，对于流媒体应用，可以允许一定程度的延迟抖动以换取更高的吞吐量；对于交互式应用，则优先保证低延迟。
- **与链路层协同：**利用链路层提供的拥塞信息（如优先级流量控制 PFC、数据中心桥接 DCB）改进拥塞检测。链路层信息可以提供更及时、更准确的拥塞指示。

- 端到端协同：在发送端和接收端之间协同优化，充分利用双向信息。接收端可以提供更丰富的反馈信息，如接收窗口通告、SACK 信息等。
- 与调度器协同：与操作系统的进程调度器、网络调度器协同，确保拥塞控制决策能够及时执行。调度优先级的调整可以减少决策延迟。

6.2.7 异构网络适配

现代网络环境日益复杂，一个 TCP 连接可能跨越多种类型的网络。Lark 算法需要适配更多的网络类型：

- 有线-无线混合网络：需要区分拥塞丢包和无线信道丢包。无线信道的丢包通常是由于信号衰落、干扰等原因，而非网络拥塞。Lark 可以通过分析丢包模式（如连续丢包 vs. 随机丢包）来区分两种情况。
- 卫星网络：需要适应极高延迟（数百毫秒）和高误码率环境。卫星网络的 BDP 很大，需要更大的窗口才能充分利用带宽。Lark 的自适应参数调优可以针对卫星网络进行特殊优化。
- 5G 网络：需要适应高带宽（Gbps 级）、低延迟（毫秒级）但可能存在突发抖动的特点。5G 网络的延迟变化可能比固定网络更大，Lark 的多信号融合机制需要对 RTT 变化更加鲁棒。
- 物联网网络：物联网设备通常资源受限（CPU、内存、电池），需要轻量级的拥塞控制算法。Lark 的基于规则决策机制比深度学习方法更适合这类场景。

6.2.8 与新兴传输协议的融合

QUIC 协议已经成为 HTTP/3 的基础，正在快速普及。将 Lark 算法与 QUIC 协议融合是重要的研究方向：

- QUIC 拥塞控制：将 Lark 的核心思想（多信号融合、差异化响应、自适应调优）移植到 QUIC 协议中。QUIC 的用户态实现使得算法的部署和更新更加灵活。
- 多路复用优化：利用 QUIC 的多流特性实现更精细的流间调度。在单个 QUIC 连接内，不同流可以有不同的优先级和拥塞控制参数。
- 0-RTT 连接优化：优化零往返时间（0-RTT）连接建立场景下的拥塞控制。0-RTT 连接需要在没有最新网络状态的情况下决定初始发送速率。

- 连接迁移支持: QUIC 支持连接迁移 (Connection Migration), 即在网络地址变化时保持连接。Lark 需要处理连接迁移后网络条件变化的情况。

6.2.9 绿色网络与能效优化

随着数据中心能耗问题日益突出, 能效成为网络设计的重要考量:

- 能耗感知拥塞控制: 在保证性能的前提下降低网络设备能耗。可以通过减少不必要的重传、优化传输调度来降低能耗。
- 动态功率管理: 根据网络负载动态调整网络设备的功率状态。在低负载时, 可以将部分链路或端口置于低功耗状态。
- 碳排放优化: 将碳排放作为优化目标之一。数据中心的碳排放与其能耗直接相关, 优化能效也是减少碳排放的有效手段。
- 可再生能源整合: 考虑数据中心可再生能源 (如太阳能、风能) 的可用性, 在可再生能源充足时增加计算和传输负载。

6.2.10 可解释人工智能在网络中的应用

虽然 Lark 采用基于规则的决策机制, 具有较好的可解释性, 但未来可以进一步研究可解释 AI 在网络拥塞控制中的应用:

- 决策可视化: 提供直观的可视化工具帮助理解和调试拥塞控制决策。可视化可以展示当前网络状态、拥塞检测结果、参数调整过程等。
- 因果分析: 分析不同拥塞信号对决策的因果影响。因果分析可以帮助理解算法行为, 发现潜在的问题和改进方向。
- 知识提取: 如果使用深度学习模型, 可以从模型中提取可解释的规则, 融入 Lark 框架。知识提取可以结合深度学习的表示能力和基于规则方法的可解释性。
- 反事实解释: 提供“如果……会怎样”的反事实解释, 帮助用户理解在不同条件下算法会如何决策。

6.3 应用场景展望

Lark 算法在以下应用场景中具有广阔的应用前景:

6.3.1 云计算数据中心

云计算数据中心承载着多种类型的工作负载，包括 Web 服务、数据库、分布式存储和机器学习训练等。Lark 的”高吞吐、低延迟”特性非常适合这种多样化的工作负载环境：

- 对于大数据分析任务，Lark 能够提供接近线速的吞吐量，加速数据处理。
- 对于在线交易系统，Lark 的低延迟特性能够满足严格的 SLA 要求。
- 对于机器学习训练，Lark 的公平性机制能够保证参数服务器和工作节点之间的高效通信。

6.3.2 边缘计算

边缘计算将计算资源部署在网络边缘，靠近用户和数据源。边缘计算场景对延迟极为敏感，Lark 的低延迟特性可以显著提升用户体验：

- 增强现实（AR）/虚拟现实（VR）应用需要毫秒级的端到端延迟。
- 自动驾驶车联网需要可靠的低延迟通信支持实时决策。
- 工业物联网（IIoT）需要低延迟和高可靠性保证生产安全。

6.3.3 内容分发网络

CDN 将内容缓存到靠近用户的节点，加速内容分发。Lark 可以优化 CDN 节点与源站、CDN 节点与用户之间的数据传输：

- 视频流媒体需要高吞吐量支持高清视频传输。
- 直播应用需要低延迟保证实时互动。
- 大文件下载需要稳定的高带宽传输。

6.3.4 高性能计算

高性能计算集群中的节点间通信对性能有严格要求。Lark 的高带宽利用率和低延迟特性可以加速并行计算应用：

- MPI 通信需要低延迟的消息传递。

- 科学计算需要高效的大规模数据交换。
- 分布式机器学习需要快速的梯度同步。

6.3.5 金融交易系统

金融交易系统对延迟极其敏感，微秒级的延迟差异可能带来巨大的经济影响。Lark 的低延迟特性非常适合这一场景：

- 高频交易需要微秒级的订单响应。
- 实时风控需要快速的数据传输和处理。
- 跨市场套利需要低延迟的跨数据中心通信。

总之，基于强化学习的网络拥塞控制是一个充满挑战和机遇的研究领域。Lark 算法在多信号融合、差异化响应和自适应调优等方面的探索，为该领域的进一步发展提供了有益的参考。我们期待未来能够看到更多创新性的研究成果，推动网络拥塞控制技术的持续进步。

6.4 研究意义与社会价值

本研究具有重要的理论意义和实际应用价值：

6.4.1 理论意义

本文将强化学习技术与网络拥塞控制相结合，在以下几个方面具有理论创新：

首先，提出了完整的 15 维观测空间设计方法，为强化学习在网络领域的应用提供了状态空间设计的参考范式。15 维观测空间涵盖了 TCP 协议栈的关键状态信息，特别是 ECN 状态和拥塞控制状态机状态，这些信息在现有方案中很少被利用。

其次，提出了多信号融合拥塞检测方法，建立了分层优先级机制和频率过滤策略的理论框架。这种方法可以推广到其他需要融合多种信号进行决策的场景。

再次，提出了差异化拥塞响应机制，揭示了不同类型拥塞信号应当采用不同响应强度的原理。这一发现对于理解和改进拥塞控制算法具有指导意义。

最后，提出了强化学习驱动的自适应参数调优机制，为解决网络协议参数自适应调整问题提供了新思路。

6.4.2 实际应用价值

本研究的成果可以直接应用于实际的数据中心网络，带来以下实际价值：

提升网络性能：Lark 算法能够将数据中心内部网络延迟降低 70% 至 81%（从 1.9~2.8ms 降至 0.4~0.8ms），同时保持高吞吐量和 99% 以上的带宽利用率，直接提升数据中心应用的性能和用户体验。

降低运营成本：更高效的拥塞控制可以提高网络资源利用率，减少网络设备的采购和运维成本。

支持新型应用：低延迟特性使得 Lark 算法能够更好地支持延迟敏感型应用，如实时分析、在线交易、边缘计算等。

6.4.3 社会影响

随着数字经济的发展，数据中心网络已经成为支撑社会运转的关键基础设施。高效的网络拥塞控制算法对于保障网络服务质量、提升社会生产效率具有重要意义。

本研究成果的推广应用，有望推动我国数据中心网络技术的进步，增强网络基础设施的核心竞争力。同时，论文的公开发表将促进学术交流，推动该领域的技术创新。

参考文献

- [1] International Data Corporation (IDC). Worldwide global datasphere forecast, 2023–2027[R]. [S.l.]: IDC, 2023.
- [2] Benson T, Akella A, Maltz D A. Network traffic characteristics of data centers in the wild[C]. Proceedings of the 10th ACM SIGCOMM Conference on Internet Measurement. [S.l.]: ACM, 2010: 267–280.
- [3] Floyd S, Henderson T, Gurtov A. The NewReno modification to TCP’s fast recovery algorithm[J]. RFC 6582, 2012.
- [4] Ha S, Rhee I, Xu L. CUBIC: a new TCP-friendly high-speed TCP variant[J]. ACM SIGOPS Operating Systems Review, 2008, 42(5): 64–74.
- [5] Brakmo L S, Peterson L L. TCP Vegas: End to end congestion avoidance on a global Internet[J]. IEEE Journal on Selected Areas in Communications, 1995, 13(8): 1465–1480.
- [6] Cardwell N, Cheng Y, Gunn C S, et al. BBR: Congestion-based congestion control[J]. ACM Queue, 2016, 14(5): 20–53.
- [7] Alizadeh M, Greenberg A, Maltz D A, et al. Data center TCP (DCTCP)[J]. ACM SIGCOMM Computer Communication Review, 2010, 40(4): 63–74.
- [8] Jay N, Rotman N, Godfrey B, et al. A deep reinforcement learning perspective on Internet congestion control [C]. International Conference on Machine Learning. [S.l.]: PMLR, 2019: 3050–3059.
- [9] Jacobson V. Congestion avoidance and control[J]. ACM SIGCOMM Computer Communication Review, 1988, 18(4): 314–329.
- [10] Xu L, Harfoush K, Rhee I. Binary increase congestion control (BIC) for fast long-distance networks[C]. IEEE INFOCOM 2004: volume 4. [S.l.]: IEEE, 2004: 2514–2524.
- [11] Gettys J, Nichols K. Bufferbloat: Dark buffers in the Internet[J]. Communications of the ACM, 2012, 55(1): 57–65.
- [12] Arun V, Balakrishnan H. Copa: Practical delay-based congestion control for the Internet[C]. 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18). [S.l.]: USENIX Association, 2018: 329–342.
- [13] Hock M, Bless R, Zitterbart M. Experimental evaluation of BBR congestion control[C]. 2017 IEEE 25th International Conference on Network Protocols (ICNP). [S.l.]: IEEE, 2017: 1–10.
- [14] Tan K, Song J, Zhang Q, et al. A compound TCP approach for high-speed and long distance networks[C]. Proceedings IEEE INFOCOM 2006. [S.l.]: IEEE, 2006: 1–12.
- [15] Liu S, Başar T, Srikant R. TCP-Illinois: A loss-and delay-based congestion control algorithm for high-speed networks[J]. Performance Evaluation, 2008, 65(6-7): 417–440.
- [16] Mascolo S, Casetti C, Gerla M, et al. TCP Westwood: Bandwidth estimation for enhanced transport over wireless links[C]. Proceedings of the 7th Annual International Conference on Mobile Computing and Networking.

- [S.l.]: ACM, 2001: 287–297.
- [17] Ramakrishnan K, Floyd S, Black D. The addition of explicit congestion notification (ECN) to IP[J]. RFC 3168, 2001.
- [18] Bai W, Chen K, Hu S, et al. Enabling ECN over generic packet scheduling[C]. Proceedings of the 12th International on Conference on emerging Networking EXperiments and Technologies. [S.l.]: ACM, 2016: 191–204.
- [19] Li Y, Miao R, Liu H H, et al. HPCC: High precision congestion control[C]. Proceedings of the ACM Special Interest Group on Data Communication. [S.l.]: ACM, 2019: 44–58.
- [20] Winstein K, Balakrishnan H. TCP ex machina: Computer-generated congestion control[J]. ACM SIGCOMM Computer Communication Review, 2013, 43(4): 123–134.
- [21] Jay N, Rotman N, Godfrey B, et al. Internet congestion control via deep reinforcement learning[C]. Advances in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2019.
- [22] Abbasloo S, Yen C Y, Chao H J. Classic meets modern: A pragmatic learning-based congestion control for the Internet[C]. Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication. [S.l.]: ACM, 2020: 632–647.
- [23] Dong M, Li Q, Zarchy D, et al. PCC: Re-architecting congestion control for consistent high performance[C]. 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15). [S.l.]: USENIX Association, 2015: 395–408.
- [24] Jain R K, Chiu D M W, Hawe W R. A quantitative measure of fairness and discrimination[R]. [S.l.]: Eastern Research Laboratory, Digital Equipment Corporation, 1984.
- [25] Finn C, Abbeel P, Levine S. Model-agnostic meta-learning for fast adaptation of deep networks[C]. International Conference on Machine Learning. [S.l.]: PMLR, 2017: 1126–1135.

附录 A 程序清单

第三章 Lark 拥塞控制算法 Python 实现 (lark-tcp 文件夹)

tcp_lark.py: Lark 强化学习智能体核心实现 (约 380 行), 包含 TcpLark 类, 实现多信号融合拥塞检测、差异化拥塞响应、自适应参数调优 (α 动态调整)、融合控制策略 (BDP 估计与窗口调整)、每流独立状态管理等全部核心算法逻辑

tcp_base.py: TCP 观测空间基类 (约 140 行), 实现 TcpEventBased (15 维事件驱动) 和 TcpTimeBased (16 维时间驱动) 两种观测空间解析模式, 负责将 ns-3 传递的原始观测向量解析为语义化的状态参数

test_lark.py: Lark 算法测试与运行脚本 (约 120 行), 基于 ns3-gym 框架的强化学习训练/评估入口, 创建 OpenGym 环境实例, 为每个 TCP 套接字 (socketUuid) 创建独立的 Lark 智能体实例, 实现多流并发控制

第三章 ns-3 网络仿真 C++ 模块 (lark-tcp 文件夹)

tcp-lark.cc / tcp-lark.h: TcpLark 拥塞控制类实现, 继承自 TcpRlBase 基类, 负责在 ns-3 TCP 协议栈中注册 Lark 算法并创建 TcpLarkEnv 环境实例

tcp-lark-env.cc / tcp-lark-env.h: Lark 专用 OpenGym 环境类 (约 230 行), 继承自 TcpGymEnv, 实现 15 维观测空间的数据采集与传递、ECN 增强的奖励计算 (ECN 惩罚因子)、延迟拥塞窗口应用机制 (Deferred Cwnd)

第四章网络仿真实验配置 (lark-tcp 文件夹)

sim.cc: ns-3 网络仿真主程序 (约 400 行), 构建哑铃型 (Dumbbell) 网络拓扑、配置 TCP 协议参数、设置 FlowMonitor 流量监控、配置 OpenGym 强化学习接口, 支持通过命令行参数灵活配置 9 种实验场景

Makefile: 自动化编译与实验运行脚本, 支持多场景批量实验、对比测试和日志收集

附录 B 攻读专业学位硕士期间撰写的论文

[1] 暂无已发表论文。

附录 C 攻读专业学位硕士期间申请的专利

[1] 暂无已申请专利。

附录 D 攻读专业学位硕士期间参加的科研项目

[1] 暂无参与项目。

致谢

感谢所有对本研究提供帮助和支持的人员。感谢 ns-3 和 OpenGym 开源社区提供的优秀仿真平台和强化学习框架。感谢审稿人提出的宝贵意见和建议。

本研究的完成离不开导师的悉心指导和实验室同学们的帮助。在研究过程中，我们进行了大量的讨论和交流，这些思想碰撞极大地丰富了研究内容。特别感谢实验室提供的计算资源，使得大规模网络仿真实验得以顺利进行。

感谢家人多年来的理解和支持，让我能够专注于科研工作。感谢所有在学习和生活中给予我帮助的老师、同学和朋友们。

最后，感谢计算机网络领域前辈们的开创性工作，正是站在巨人的肩膀上，本研究才得以完成。Van Jacobson、Sally Floyd 等先驱在 TCP 拥塞控制领域的贡献，为后来的研究奠定了坚实的基础。我们希望本研究能够在他们开创的道路上继续前行，为网络拥塞控制技术的发展做出微薄的贡献。